

Agentic AI on Azure

Enterprise Master Class - Course Book

12 weekly labs + framework comparison & Azure data-service projects
Compiled from the GitHub repositories (github.com/satyajeetaiml-hue)

Table of Contents

Table of Contents	2
Part I - Course Overview	3
Course Overview & Reference Architecture	4
Part II - Weekly Labs	9
Week 1 - Foundations of Agentic AI	10
Week 2 - Microsoft Foundry & Agent Service	14
Weeks 3-4 - Microsoft Agent Framework	20
Week 5 - Tools, MCP & Interoperability	25
Weeks 6-7 - Multi-Agent Orchestration	30
Week 8 - Memory, State & Grounding (RAG)	34
Week 9 - Hosting, Deployment & Scale	40
Week 10 - Observability, Evaluation & Governance	44
Week 11 - Enterprise Security & Compliance	48
Week 12 - Capstone: Build, Demo & Review	53
Part III - Framework Comparison Projects	57
LangGraph on Azure - Support Triage Graph	58
Semantic Kernel on Azure - Claims Validation	62
Part IV - Semantic Kernel + Azure Data Services	66
SK + Azure PostgreSQL - Orders Assistant	67
SK + Azure AI Search - Knowledge Assistant	70
SK + Azure Cosmos DB - Conversation Memory	73

Part I - Course Overview

Course Overview & Reference Architecture

Agentic AI on Azure - Enterprise Master Class (Starter Project)

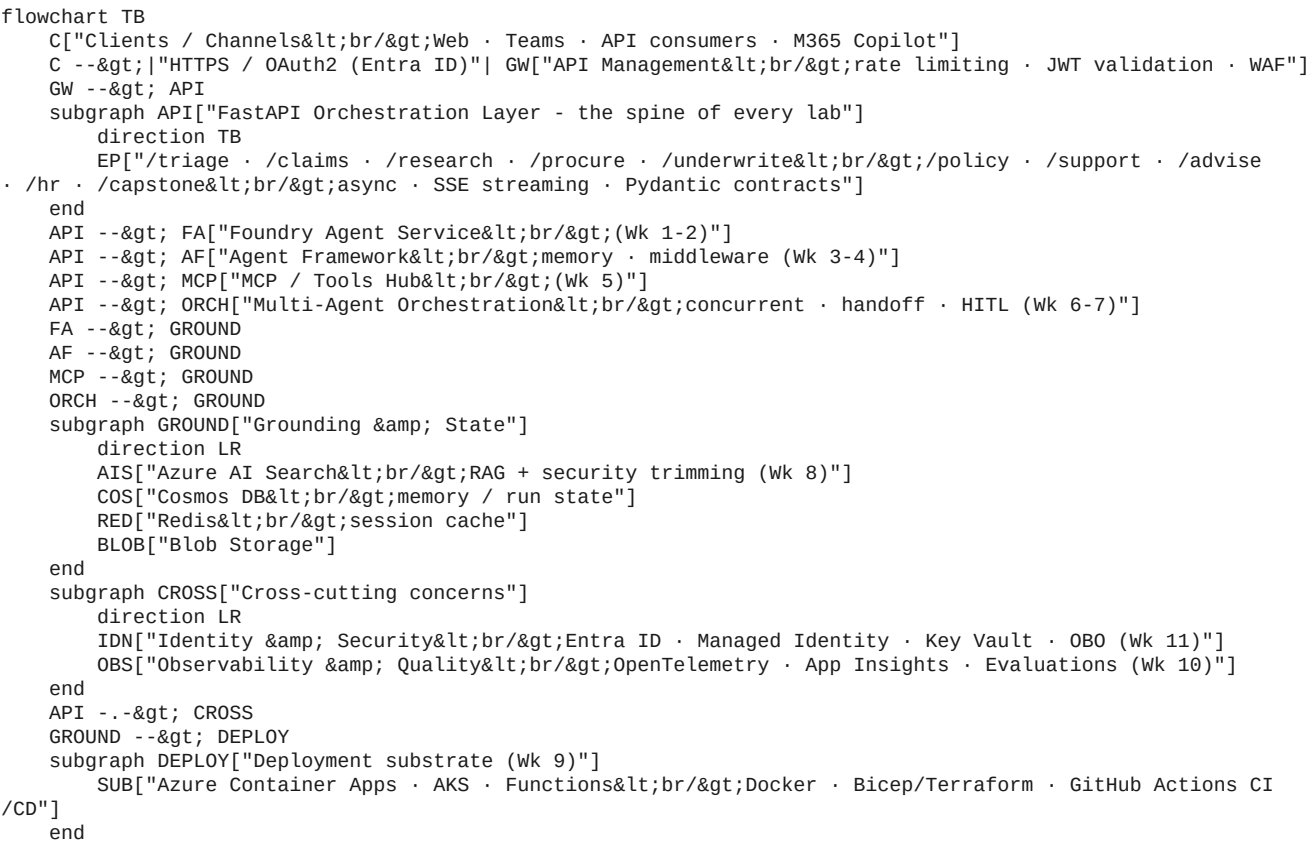
A production-oriented **FastAPI** starter for building agentic AI systems on Azure, structured to follow the **12-Week Enterprise Master Class** (see docs/Course-Enterprise-UseCases.md). The project is designed so it **runs locally out of the box** (Week 1 "hello agent" works with a built-in mock reasoning loop - no Azure keys required), and progressively layers in real Azure services week by week.

Stack: **Python 3.11+** · **FastAPI** · **Pydantic v2** · **Uvicorn** · *Microsoft Foundry* · *Microsoft Agent Framework* ·
MCP · *A2A* · *Azure AI Search* · *Cosmos DB* · *Container Apps*.

Architecture

The "north star" you build toward across the 12 weeks. Each block is annotated with the week(s) that implement it. (A plain-text version is in docs/Course-Enterprise-UseCases.md.)

[mermaid]



Lab index - one repo per week

This repo is the **course hub**. Each week is also published as a **standalone, runnable repo**.
[x] **All 12 weeks are fully built out** - every lab ships a runnable offline **mock backend** (deterministic, fully tested) plus a lazy-imported **real Azure backend** selected by env vars, with typed schemas, hermetic tests, a detailed README, Dockerfile, and green CI. ****61 tests passing across all repos.** **** Clone any week and run `uvicorn app.main:app --reload`.**

Week	Lab repo	Enterprise use case	Key built-out feature	Status
------	----------	---------------------	-----------------------	--------

1	week01-foundations	IT Helpdesk Triage Agent	reason->plan->act->observe loop + Foundry Responses	[x] Built out
2	week02-foundry-claims	Insurance Claims Intake	Foundry Agent Service function tool loop	[x] Built out
3-4	week03-04-agent-framework	Wealth Research Assistant	memory + middleware (PII/disclaimer) + tools	[x] Built out
5	week05-mcp-tools	Procurement Operations Agent	tool registry + idempotency + MCP discovery	[x] Built out
6-7	week06-07-multi-agent	Loan Underwriting Pipeline	concurrent agents + human-in-the-loop resume	[x] Built out
8	week08-rag-grounding	Clinical Policy Assistant (RAG)	retrieval + security trimming + citations	[x] Built out
9	week09-hosting-scale	Customer Service Swarm	durable background tasks + status polling	[x] Built out
10	week10-observability	Advice Quality Monitoring	eval scoring + metrics/traces + OTel	[x] Built out
11	week11-security	HR Self-Service (least privilege)	Entra JWT + OBO + RBAC + audit	[x] Built out
12	week12-capstone	Capstone onboarding pipeline	staged multi-agent pipeline	[x] Built out

Each lab endpoint is listed in its own README. Backends run in **mock mode** with no Azure; set the documented env vars (+ `az login`) to switch to the real Azure backend.

Browse them all: <https://github.com/satyajeetaiml-hue?tab=repositories&q=agentic-ai-azure>

Other agent frameworks (bonus projects)

The labs use the **Microsoft Agent Framework / Foundry**. These companion repos solve the same kind of

problem with the two other major frameworks, so you can compare APIs side by side. Both run **for real offline** (the framework executes; Azure OpenAI is optional) and have green CI.

Framework	Repo	What it shows
LangGraph	agentic-ai-azure-langgraph	A StateGraph triage agent - nodes + conditional edges + state; optional Azure OpenAI node

Semantic Kernel	agentic-ai-azure-semantic-kernel	A Kernel + native plugin (@kernel_function); optional Azure automatic function calling
------------------------	----------------------------------	--

COMPARISON.md - Microsoft Agent Framework vs. LangGraph vs. Semantic Kernel: abstractions, control flow, state, tools, Azure fit, the same agent in each, and how to choose.

Semantic Kernel + Azure data services

Focused Semantic Kernel projects pairing a real SK kernel + native plugin with one Azure data store (each runs offline with an in-memory store; the Azure service is the optional backend):

Azure service	Repo	Use case
Azure Database for PostgreSQL	agentic-ai-azure-sk-postgresql	Orders Assistant - plugin queries a relational orders table
Azure AI Search	agentic-ai-azure-sk-ai-search	Knowledge Assistant (RAG) - plugin retrieves + cites sources
Azure Cosmos DB	agentic-ai-azure-sk-cosmosdb	Durable conversation memory - plugin remembers/recalls turns

Quick start

[bash]

```
# 1. Create and activate a virtual environment
python -m venv .venv
# Windows (PowerShell):
.\.venv\Scripts\Activate.ps1
# macOS/Linux:
# source .venv/bin/activate
# 2. Install dependencies
pip install -r requirements.txt
# 3. Copy the env template (optional - app runs in MOCK mode without it)
copy .env.example .env           # Windows
# cp .env.example .env           # macOS/Linux
# 4. Run the API
uvicorn app.main:app --reload
```

Then open:

- Swagger UI: <http://127.0.0.1:8000/docs>
- Health check: <http://127.0.0.1:8000/health>
- Week 1 hello-agent: POST <http://127.0.0.1:8000/api/v1/hello>

Example request:

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/hello ^
-H "Content-Type: application/json" ^
-d '{"message": "My laptop won't connect to VPN and it's urgent"}'
```

Project layout

```
azure-agentic-ai-masterclass/
app/
  main.py          # FastAPI application + router registration
  config.py        # Pydantic settings (reads .env)
  core/
    agent_loop.py  # reason -> plan -> act -> observe loop (mock + Azure-ready)
    logging.py     # structured logging setup
  routers/
    hello.py       # Week 1 - IT Helpdesk Triage Agent
    health.py      # liveness/readiness
  schemas/
    agent.py       # Pydantic request/response contracts
  labs/            # one folder per course week with its own README & TODOs
    week01_foundations/
    week02_foundry/
    ...
```

```
docs/
  Course-Enterprise-UseCases.md # the full course companion
infra/                          # Bicep IaC placeholders (Container Apps, etc.)
tests/                          # pytest suite
.github/workflows/ci.yml
Dockerfile
docker-compose.yml
requirements.txt
pyproject.toml
```

Running with Docker

[*bash*]

```
docker build -t azure-agentic-ai .
docker run -p 8000:8000 azure-agentic-ai
```

Or with compose:

[*bash*]

```
docker compose up --build
```

Configuration

All settings live in `app/config.py` and are read from environment variables / `.env`.

See `.env.example` for the full list. If `AZURE_OPENAI_ENDPOINT` is **not** set, the agent loop falls back to a deterministic **mock** so the app stays runnable for learning.

Variable	Purpose
APP_ENV	local / dev / prod
AZURE_OPENAI_ENDPOINT	Azure OpenAI / Foundry endpoint (enables real model calls)
AZURE_OPENAI_API_KEY	Key (prefer Managed Identity in Azure)
AZURE_OPENAI_DEPLOYMENT	Chat model deployment name

The 12-week path

Each `labs/weekNN_*` folder maps to a week in the course companion and contains a `README.md` with the learning goal, enterprise use case, and a hands-on TODO checklist. Start at `labs/week01_foundations`.

Testing

[*bash*]

```
pytest -q
```

License

MIT - see LICENSE.

Teaching slides

Every repo ships a **7-slide deck** (`.pptx`, with presenter notes) and a **PDF handout** (slides + notes).

This is the master index - links download the file directly.

Slides per deck: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

This hub: Deck · Handout (PDF)

Weekly labs

Week	Download
1 - Foundations	Deck · PDF

2 - Foundry Claims	Deck · PDF
3-4 - Agent Framework	Deck · PDF
5 - MCP & Tools	Deck · PDF
6-7 - Multi-Agent	Deck · PDF
8 - RAG	Deck · PDF
9 - Hosting & Scale	Deck · PDF
10 - Observability	Deck · PDF
11 - Security	Deck · PDF
12 - Capstone	Deck · PDF

Framework comparison projects

Project	Download
LangGraph	Deck · PDF
Semantic Kernel	Deck · PDF

Semantic Kernel + Azure data services

Project	Download
SK + PostgreSQL	Deck · PDF
SK + AI Search	Deck · PDF
SK + Cosmos DB	Deck · PDF

Part II - Weekly Labs

Week 1 - Foundations of Agentic AI

Week 1 - Foundations of Agentic AI

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no `az login`, keys, or `.env` required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Understand the **agent loop** (reason -> plan -> act -> observe), tool use, and the agent contract.

Enterprise use case - "IT Helpdesk Triage Agent" (Cross-industry)

A single agent reads a support ticket, reasons about category/severity, and either answers from a knowledge base (a **tool**) or escalates to a human (a **guardrail**).

[x] What this repo implements

- **Mock backend** - deterministic reason->plan->act->observe loop with a KB-lookup tool and escalation rule. Runs offline and is fully tested.
- **Foundry backend** - uses Microsoft Foundry (azure-ai-projects v2, Responses API) to classify the ticket, grounded by the KB. Lazy-imported, selected when `FOUNDRY_PROJECT_ENDPOINT` is set.

Condition	Backend
<code>FOUNDRY_PROJECT_ENDPOINT</code> unset	mock (default)
<code>FOUNDRY_PROJECT_ENDPOINT</code> set + <code>az login</code>	foundry

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1 # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/triage \
-H "Content-Type: application/json" \
-d '{"message": "Production is down, this is urgent!"}'
```

Run tests: `pytest -q`

Foundry mode

`az login`, then set in `.env`:

```
FOUNDRY_PROJECT_ENDPOINT=https://&lt;account&gt;.services.ai.azure.com/api/projects/&lt;project&gt;
FOUNDRY_MODEL_NAME=gpt-4o
```

`GET /health` will report `"backend": "foundry"`.

Architect's lens

- When is an **agent** right vs. a deterministic workflow or single LLM call? (cost, latency, auditability)
- Foundry vs. Agent Framework vs. raw model calls.
- The **agent contract**: inputs, tools, guardrails, success criteria.

Tech stack

FastAPI, Pydantic v2, Microsoft Foundry (azure-ai-projects v2, Responses API), azure-identity.

Structure

```
app/service.py    # settings + schemas + mock & foundry backends
app/main.py       # FastAPI app + POST /api/v1/triage
tests/test_app.py
```

Series

Next: Week 2 - Foundry Claims Intake.

All labs: search `agentic-ai-azure`.

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-week01-foundations.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-week01-foundations-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `service.py`

```
"""Week 1 - Foundations: the IT Helpdesk Triage Agent.
Demonstrates the agent loop (reason -> plan -> act -> observe) with one tool
(a mock KB lookup) and an escalation guardrail. Ships two backends:
* ``MockTriageBackend`` - deterministic heuristic, runs offline (default, tested).
* ``FoundryTriageBackend`` - uses Microsoft Foundry (azure-ai-projects v2,
  Responses API) to reason about the ticket, lazy-imported so Azure is optional.
"""
from __future__ import annotations
import json
from functools import lru_cache
from typing import Any
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    foundry_project_endpoint: str = ""
    foundry_model_name: str = "gpt-4o"
    @property
    def use_foundry(self) -> bool:
        return bool(self.foundry_project_endpoint)
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class TriageRequest(BaseModel):
    message: str = Field(..., min_length=1, description="Incoming support ticket / user message.")
class AgentStep(BaseModel):
    phase: str
    detail: str
class TriageResponse(BaseModel):
    answer: str
    category: str
    severity: str
    escalated: bool
    mode: str
    steps: list[AgentStep]
# shared knowledge / rules
KB: dict[str, str] = {
    "vpn": "Reset your VPN client, then reconnect with your corporate credentials.",
    "password": "Use the self-service portal to reset your password; locked accounts auto-unlock in 30 min.",
    "email": "Restart Outlook and re-add the account; check service health if many users are affected.",
}
ESCALATE_SIGNALS = ("urgent", "down", "outage", "breach", "production", "asap")
```

```

SYSTEM_INSTRUCTIONS = (
    "You are an IT helpdesk triage agent. Classify the ticket and decide how to handle it. "
    "Return ONLY JSON with keys: category (one of vpn, password, email, general), "
    "severity (normal|high), escalated (boolean), answer (string). "
    "Escalate (escalated=true) when the issue sounds urgent or affects production/many users."
)

def _kb_lookup(text: str) -> tuple[str, str | None]:
    low = text.lower()
    for key, val in KB.items():
        if key in low:
            return key, val
    return "general", None

# mock backend
class MockTriageBackend:
    mode = "mock"
    def triage(self, message: str) -> TriageResponse:
        steps: list[AgentStep] = []
        severity = "high" if any(s in message.lower() for s in ESCALATE_SIGNALS) else "normal"
        steps.append(AgentStep(phase="reason", detail=f"Severity assessed as '{severity}'."))
        plan = "escalate" if severity == "high" else "answer_from_kb"
        steps.append(AgentStep(phase="plan", detail=f"Strategy: {plan}."))
        category, answer = _kb_lookup(message)
        steps.append(AgentStep(phase="act", detail=f"KB lookup category='{category}', hit={answer is not None}."))
        if plan == "escalate":
            final, escalated = "Escalating to a human engineer (time-sensitive).", True
        elif answer:
            final, escalated = answer, False
        else:
            final, escalated = "No KB match; routing to the helpdesk queue.", True
        steps.append(AgentStep(phase="observe", detail=f"escalated={escalated}."))
        return TriageResponse(
            answer=final, category=category, severity=severity,
            escalated=escalated, mode=self.mode, steps=steps,
        )

# foundry backend
class FoundryTriageBackend:
    mode = "foundry"
    def triage(self, message: str) -> TriageResponse:
        steps = [AgentStep(phase="reason", detail="Asked Foundry model to classify the ticket.")]
        data = _foundry_json(SYSTEM_INSTRUCTIONS, f"Ticket: {message}")
        category = data.get("category") or "general"
        severity = data.get("severity") or "normal"
        escalated = bool(data.get("escalated"))
        # Ground the answer with the KB when available.
        _, kb_answer = _kb_lookup(message)
        answer = data.get("answer") or kb_answer or "Routing to the helpdesk queue."
        steps.append(AgentStep(phase="act", detail=f"Model returned category='{category}'."))
        steps.append(AgentStep(phase="observe", detail=f"escalated={escalated}."))
        return TriageResponse(
            answer=answer, category=category, severity=severity,
            escalated=escalated, mode=self.mode, steps=steps,
        )

def _foundry_json(system: str, user: str) -> dict[str, Any]:
    """Call the Foundry project's Responses API and parse a JSON reply."""
    from azure.ai.projects import AIProjectClient
    from azure.identity import DefaultAzureCredential
    s = get_settings()
    with (
        DefaultAzureCredential() as cred,
        AIProjectClient(endpoint=s.foundry_project_endpoint, credential=cred) as proj,
    ):
        client = proj.get_openai_client()
        resp = client.responses.create(
            model=s.foundry_model_name,
            input=[{"role": "system", "content": system}, {"role": "user", "content": user}],
        )
        text = (resp.output_text or "{}").strip()
    if text.startswith("```"):
        text = text.strip("`").removeprefix("json").strip()
    start, end = text.find("{", text.rfind("}")
    try:
        return json.loads(text[start : end + 1] if start != -1 else text)
    except json.JSONDecodeError:
        return {}

def get_backend():
    return FoundryTriageBackend() if get_settings().use_foundry else MockTriageBackend()

```

» [main.py](#)

```

"""Week 1 - Foundations of Agentic AI.
IT Helpdesk Triage Agent. Runs in MOCK mode out of the box; set
FOUNDRY_PROJECT_ENDPOINT (+ `az login`) for the Foundry backend.
Run: uvicorn app.main:app --reload
"""

from fastapi import FastAPI
from app.service import TriageRequest, TriageResponse, get_backend, get_settings
settings = get_settings()
app = FastAPI(title="Week 1 - Foundations (Triage Agent)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "week": "1",
        "backend": "foundry" if settings.use_foundry else "mock",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week01-foundations",
        "endpoint": "/api/v1/triage",
        "backend": "foundry" if settings.use_foundry else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/triage", response_model=TriageResponse, tags=["week01"])
def triage(payload: TriageRequest) -> TriageResponse:
    return get_backend().triage(payload.message)

```

Week 2 - Microsoft Foundry & Agent Service

Week 2 - Microsoft Foundry & Foundry Agent Service

> **Run in VS Code - no Azure needed.** pip install -r requirements.txt, then uvicorn app.main:app --reload and open <http://127.0.0.1:8000/docs>. Runs in **mock mode** by default - no az login, keys, or .env required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class** (12 weeks).

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Create a hosted agent on **Microsoft Foundry Agent Service** and expose it through a FastAPI REST endpoint - secured with **Managed Identity / Entra ID** (no API keys in code).

Enterprise use case - "Insurance Claims Intake Agent" (Insurance)

A customer submits a free-text claim. The agent:

1. **Extracts** structured fields (claimant, policy number, incident date, claim type, amount),
2. **Validates** the policy by calling a `policy_lookup` tool (your policy system),
3. **Decides** an outcome - create a case, send for review, or reject - and returns a typed result.

[x] What this repo implements

Unlike the other week starters (which ship a single mock stub), **this lab is built out:**

- A real **Foundry Agent Service backend** that creates a **persistent hosted agent** with the current `azure-ai-projects v2` SDK (`agents.create_version + PromptAgentDefinition`) - it's registered in your Foundry project (visible in the portal), reused across requests, and runs via the Responses API with an `agent_reference`, calling a custom **function tool** (`policy_lookup`) during the run.
- A deterministic **mock backend** that mirrors the same extract -> tool-call -> decide flow, so the service is **runnable and fully testable with zero Azure resources**.
- Typed **Pydantic contracts** as the trust boundary, and a clean `ClaimsAgent` interface that swaps backends based on config.

The backend is chosen automatically:

Condition	Backend
FOUNDRY_PROJECT_ENDPOINT unset	mock (default - offline, deterministic)
FOUNDRY_PROJECT_ENDPOINT set + az login	foundry (real Agent Service)

Quick start (mock mode - no Azure)

[bash]

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1      # Windows (macOS/Linux: source .venv/bin/activate)
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Open <http://127.0.0.1:8000/docs>. Try it:

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/claims/intake \
-H "Content-Type: application/json" \
-d '{"claim_text": "Car accident on 2026-06-03, policy POL-12345, front bumper damaged, approx $1,200."}'
```

Sample response:

[json]

```
{
```

```

"case_id": "CASE-9F3A1C7D",
"decision": "created",
"summary": "Claim validated and a case was created.",
"extracted": {
  "policy_number": "POL-12345",
  "incident_date": "2026-06-03",
  "claim_type": "auto",
  "estimated_amount": 1200.0
},
"policy": {
  "policy_number": "POL-12345",
  "valid": true,
  "status": "active",
  "holder": "Jordan Avery",
  "coverage": ["collision", "liability", "roadside"],
  "deductible": 500.0
},
"mode": "mock",
"agent_steps": ["Extracted fields ...", "Called policy_lookup -& status=active.", "Decision: creat
ed."]
}

```

Test policy numbers: POL-12345 / POL-67890 (active -> *created*), POL-00001 (lapsed -> *rejected*), anything else (not found -> *needs_review*).

Run the tests:

```

[ bash ]
    pytest -q

```

Switching to the real Foundry Agent Service

1. Provision (one-time)

- Create a **Microsoft Foundry** project - see

Create a project.

- Deploy a chat model (e.g. **gpt-4o**) in that project; note its **deployment name**.
- Grant yourself the **Azure AI User** role on the project resource (Access Control / IAM).

See RBAC in Foundry.

2. Authenticate (Managed Identity / Entra ID - no keys)

This project uses `DefaultAzureCredential`, so locally just:

```

[ bash ]
    az login

```

In Azure (Container Apps/AKS), assign a **Managed Identity** the same role - the *exact same code* works with no secrets, which is the point of this week.

3. Configure

```

[ bash ]
    cp .env.example .env

```

Set in `.env`:

```

FOUNDRY_PROJECT_ENDPOINT=https://&your-account&.services.ai.azure.com/api/projects/&your-project&
FOUNDRY_MODEL_NAME=gpt-4o          # your deployment name

```

4. Run

```

[ bash ]
    uvicorn app.main:app --reload

```

On first request the backend **creates a hosted agent** named by `FOUNDRY_AGENT_NAME` (default `claims-intake-agent`) via `project.agents.create_version(...)` - it appears in your Foundry project's

Agents list - then reuses it for subsequent requests. GET `/health` reports `"backend": "foundry"`, and POST `/api/v1/claims/intake` runs the hosted agent, which calls `policy_lookup` during the run.

***How it's wired** (app/agent/foundry_agent.py): the hosted agent is defined with PromptAgentDefinition(model, instructions, tools=[FunctionTool(...)]) and created via agents.create_version. Each request runs it with `openai\_client.responses.create(input=..., extra\_body={"agent\_reference": {"name": ..., "type": "agent\_reference"}}); function\_call` items are executed locally and fed back as function_call_output. This is the verified **azure-ai-projects v2** Agent Service API (the older 1.x create_agent + threads/runs flow is deprecated).*

>

In production, create the agent once via IaC/CI and reference it by name (don't create per process).

The lab (extend it)

1. [x] Wrap a Foundry agent behind a FastAPI /claims/intake endpoint *(done - use as your reference)*.
2. [x] Add a policy-lookup tool the agent calls during the run *(done - app/tools/policy_lookup.py)*.
3. [x] Secure the connection with Managed Identity / DefaultAzureCredential *(done)*.
4. [x] Create the agent as a **persistent hosted agent** (appears in the Foundry portal) and reuse it across requests *(done - _ensure_agent in app/agent/foundry_agent.py)*.
5. Replace the in-memory policy DB with a real call to your policy API (via APIM + Managed Identity).
6. Add a second tool (e.g. coverage_check) and a guardrail for low-confidence extractions.

Architect's lens

- **Hosted vs. self-orchestrated agents** - when the managed thread/state/tool-call runtime saves ops effort.
- **Quota, model routing, region selection** for data residency (claims data is regulated).
- **Identity:** Managed Identity end-to-end; the same code runs locally (az login) and in Azure (MI) with no secrets - secrets that **do** exist (downstream APIs) belong in **Key Vault**.
- **Contracts as the trust boundary:** the model's output is re-validated against the policy system before a decision is finalized (see _build_response - we never trust the model's tool echo).

Tech stack

Microsoft Foundry Agent Service, azure-ai-projects v2 (Responses API), azure-identity (Managed Identity / Entra ID), FastAPI, Pydantic v2. Key Vault for downstream secrets.

Project structure

```

agent-ai-azure-week02-foundry-claims/
app/
  main.py           # FastAPI app
  config.py         # settings (FOUNDRY_* env) + backend selection
  schemas.py        # Pydantic contracts (request/response)
  routers/claims.py # POST /api/v1/claims/intake
agent/
  base.py           # ClaimsAgent interface, system prompt, decision rule
  mock_agent.py     # deterministic offline backend
  foundry_agent.py  # Foundry Agent Service (Responses + function tool)
  tools/policy_lookup.py # the policy-system tool (fn + JSON schema)
tests/test_claims.py # hermetic tests (mock backend)
requirements.txt
Dockerfile
.env.example
README.md

```

Where this fits

Part of **Week 2** of the course series.

Previous: Week 1 - Foundations.
Next: Weeks 3-4 - Agent Framework.

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-week02-foundry-claims.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-week02-foundry-claims-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `foundry_agent.py`

```
"""Foundry Agent Service backend - **persistent hosted agent**.  
Uses the current **azure-ai-projects v2** SDK to create a *hosted* agent that is  
registered in your Foundry project (visible in the portal) and reused across  
requests, then runs it via the Responses API with an ``agent_reference``:  
* ``project.agents.create_version(agent_name, definition=PromptAgentDefinition(  
    model, instructions, tools=[FunctionTool(...)]))`` - the hosted agent.  
* ``openai_client.responses.create(input=..., extra_body={"agent_reference": ...})``  
  - runs the hosted agent; tool calls surface as ``function_call`` output items  
  which we execute (``policy_lookup``) and feed back as ``function_call_output``.  
The flow still mirrors the agent loop (reason -&gt; plan -&gt; act -&gt; observe). Azure SDK  
imports are lazy so the mock backend and tests run without the Azure packages.  
"""  
  
import json  
import uuid  
from typing import Any  
from app.agent.base import SYSTEM_INSTRUCTIONS, decide_from_policy  
from app.config import get_settings  
from app.schemas import (  
    ClaimIntakeRequest,  
    ClaimIntakeResponse,  
    ExtractedClaim,  
    PolicyValidation,  
)  
from app.tools.policy_lookup import POLICY_TOOL_SCHEMA, policy_lookup  
_MAX_TOOL_ITERATIONS = 5  
# Process-level cache so the hosted agent is created once and reused across requests.  
_AGENT_VERSIONS: dict[str, str] = {}  
class FoundryClaimsAgent:  
    def __init__(self) -&gt; None:  
        self.settings = get_settings()  
    def intake(self, request: ClaimIntakeRequest) -&gt; ClaimIntakeResponse:  
        try:  
            from azure.ai.projects import AIProjectClient  
            from azure.identity import DefaultAzureCredential  
        except ImportError as exc: # pragma: no cover - depends on optional deps  
            raise RuntimeError(  
                "Foundry backend requires 'azure-ai-projects' and 'azure-identity'. "  
                "Install requirements.txt, or unset FOUNDRY_PROJECT_ENDPOINT to use mock mode."  
            ) from exc  
        s = self.settings  
        with (  
            DefaultAzureCredential() as credential,  
            AIProjectClient(endpoint=s.foundry_project_endpoint, credential=credential) as project,  
            project.get_openai_client() as openai_client,  
        ):  
            agent_name = self._ensure_agent(project)  
            final_text, steps = self._run(openai_client, agent_name, request)  
            return self._build_response(request, final_text, steps)  
    def _ensure_agent(self, project) -&gt; str:  
        """Create the hosted agent once (cached), returning its name to reference."""  
        from azure.ai.projects.models import FunctionTool, PromptAgentDefinition  
        name = self.settings.foundry_agent_name  
        if name in _AGENT_VERSIONS:  
            return name  
        tool = FunctionTool(  
            name="policy_lookup",
```

```

        description=POLICY_TOOL_SCHEMA["description"],
        parameters=POLICY_TOOL_SCHEMA["parameters"],
        strict=True,
    )
    agent = project.agents.create_version(
        agent_name=name,
        definition=PromptAgentDefinition(
            model=self.settings.foundry_model_name,
            instructions=SYSTEM_INSTRUCTIONS,
            tools=[tool],
        ),
    )
    _AGENT_VERSIONS[name] = agent.version
    return name
def _run(self, openai_client, agent_name: str, request: ClaimIntakeRequest):
    """Drive the hosted agent + function-tool loop; return (final_text, steps)."""
    steps = [f"Using hosted Foundry agent '{agent_name}'."]
    ref = {"agent_reference": {"name": agent_name, "type": "agent_reference"}}
    response = openai_client.responses.create(input=self._user_prompt(request), extra_body=ref)
    for _ in range(_MAX_TOOL_ITERATIONS):
        calls = [it for it in response.output if getattr(it, "type", None) == "function_call"]
        if not calls:
            break
        tool_outputs = []
        for call in calls:
            args = json.loads(call.arguments or "{}")
            if call.name == "policy_lookup":
                result = policy_lookup(args.get("policy_number", ""))
                steps.append(f"Agent called policy_lookup -&gt; status={result.get('status')}")
            else:
                result = {"error": f"unknown tool '{call.name}'"}
            tool_outputs.append(
                {
                    "type": "function_call_output",
                    "call_id": call.call_id,
                    "output": json.dumps(result),
                }
            )
        response = openai_client.responses.create(
            input=tool_outputs, previous_response_id=response.id, extra_body=ref
        )
    return (response.output_text or "{}"), steps
@staticmethod
def _user_prompt(request: ClaimIntakeRequest) -&gt; str:
    hint = f"\nKnown policy number: {request.policy_number}" if request.policy_number else ""
    return f"Process this claim:\n\n{request.claim_text}{hint}"
def _build_response(
    self, request: ClaimIntakeRequest, final_text: str, steps: list[str]
) -&gt; ClaimIntakeResponse:
    data = self._parse_json(final_text)
    extracted = ExtractedClaim(
        claimant_name=data.get("claimant_name"),
        policy_number=data.get("policy_number") or request.policy_number,
        incident_date=data.get("incident_date"),
        claim_type=data.get("claim_type"),
        description=data.get("description") or request.claim_text,
        estimated_amount=_to_float(data.get("estimated_amount")),
    )
    # Re-validate the policy authoritatively (don't trust the model's tool echo).
    if extracted.policy_number:
        policy = PolicyValidation(**policy_lookup(extracted.policy_number))
    else:
        policy = PolicyValidation(valid=False, status="missing", message="No policy number provided.")
    decision = decide_from_policy(policy.valid, policy.status, bool(extracted.policy_number))
    steps.append(f"Validated policy and finalized decision: {decision}.")
    return ClaimIntakeResponse(
        case_id=f"CASE-{uuid.uuid4().hex[:8].upper()}",
        decision=decision,
        summary=data.get("summary") or "Claim processed by the Foundry hosted agent.",
        extracted=extracted,
        policy=policy,
        mode="foundry",
        agent_steps=steps,
    )
@staticmethod
def _parse_json(text: str) -&gt; dict[str, Any]:
    """Best-effort extraction of the JSON object from the model's reply."""
    cleaned = text.strip()
    if cleaned.startswith("`"):

```

```

        cleaned = cleaned.split("```")[1] if "```" in cleaned[3:] else cleaned[3:]
        cleaned = cleaned.removeprefix("json").strip()
        start, end = cleaned.find("{", cleaned.rfind("}")
        if start != -1 and end != -1:
            cleaned = cleaned[start : end + 1]
        try:
            return json.loads(cleaned)
        except json.JSONDecodeError:
            return {}
def _to_float(value: Any) -> float | None:
    if value is None:
        return None
    try:
        return float(str(value).replace(",", "").replace("$", "").strip())
    except (ValueError, TypeError):
        return None

```

» main.py

```

"""Week 2 - Microsoft Foundry & Foundry Agent Service.
Insurance Claims Intake Agent exposed as a FastAPI service. Runs in MOCK mode
out of the box; set FOUNDRY_PROJECT_ENDPOINT (+ `az login`) to use the real
Foundry Agent Service backend. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.config import get_settings
from app.routers import claims
settings = get_settings()
app = FastAPI(
    title="Week 2 - Foundry Claims Intake Agent",
    description="Insurance Claims Intake Agent on Microsoft Foundry Agent Service.",
    version="0.2.0",
)
app.include_router(claims.router)
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "week": "2",
        "use_case": "Insurance Claims Intake Agent",
        "backend": "foundry" if settings.use_foundry else "mock",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week02-foundry-claims",
        "endpoint": "/api/v1/claims/intake",
        "backend": "foundry" if settings.use_foundry else "mock",
        "docs": "/docs",
    }

```

Weeks 3-4 - Microsoft Agent Framework

Weeks 3-4 - Microsoft Agent Framework

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no `az login`, keys, or `.env` required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Use Agent Framework concepts - **memory**, **middleware**, and **grounding tools** - behind a FastAPI service.

Enterprise use case - "Wealth Management Research Assistant" (Financial Services)

An advisor asks for a portfolio summary. The agent pulls **holdings** and **market data** (tools), keeps **session memory**, redacts PII on the way in, and injects a compliance **disclaimer** on the way out.

[x] What this repo implements

- **Memory** - per-session_id conversation memory (in-memory mock; **Cosmos DB** in prod).
- **Middleware** - input **PII redaction** (email/SSN/phone) + output **disclaimer injection**.
- **Grounding tools** - `get_holdings` + `get_market_data`.
- **Mock backend** (offline, tested) and **Foundry backend** (azure-ai-projects v2 Responses API).

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/research \
-H "Content-Type: application/json" \
-d '{"advisor_query": "Summarize the Henderson portfolio", "session_id": "demo"}'
```

Run tests: `pytest -q`. Repeated calls with the same `session_id` grow `memory_turns`.

Foundry mode

`az login`, then set `FOUNDRY_PROJECT_ENDPOINT` + `FOUNDRY_MODEL_NAME` in `.env`.

Wire durable memory (Azure Cosmos DB)

Session memory is pluggable: **in-memory** by default, **Cosmos DB** when a connection string is set (`app/service.py -> get_memory_store()`). Memory is partitioned by `sessionId`, so conversations survive

restarts and scale across instances.

1. Provision (Azure CLI)

[bash]

```
az login
RG=rg-agentic-mem
az group create -n $RG -l eastus
az cosmosdb create -g $RG -n my-agent-cosmos --default-consistency-level Session
az cosmosdb keys list -g $RG -n my-agent-cosmos --type connection-strings \
--query "connectionStrings[0].connectionString" -o tsv # copy this
```

The app **auto-creates** the database (agentmemory) and container (sessions, partition key

/sessionId) on first use - no manual schema step.

2. Configure .env

```
COSMOS_CONNECTION_STRING=&lt;connection-string&gt;
COSMOS_DATABASE=agentmemory
COSMOS_CONTAINER=sessions
```

3. Run - memory is now durable

[bash]

```
uvicorn app.main:app --reload # GET /health -&gt; "memory": "cosmos"
```

[bash]

```
# Same session_id across calls -&gt; memory_turns keeps growing, even after a restart
curl -X POST http://127.0.0.1:8000/api/v1/research \
  -H "Content-Type: application/json" \
  -d '{"advisor_query": "Summarize the Henderson portfolio", "session_id": "advisor-42"}'
```

Each turn is upserted as an item; memory_turns is a SELECT VALUE COUNT(1) scoped to the partition, and

recent context is a partition-scoped ORDER BY c.ts DESC query.

Architect's lens

- Memory tiers: session (Redis) vs. durable (Cosmos) vs. semantic (AI Search) - cost & consistency.
- Partition strategy: /sessionId keeps each conversation's reads/writes single-partition (cheap & fast).
- Middleware as a **governance seam** - enforce policy without touching agent logic.

Tech stack

Microsoft Agent Framework concepts, **Azure Cosmos DB** (durable memory), Content Safety (filtering), FastAPI, azure-ai-projects v2 (Responses API).

Structure

```
app/service.py # settings, schemas, memory stores (in-memory + Cosmos), middleware, tools, backends
app/main.py # POST /api/v1/research
tests/test_app.py # mock backend + middleware + memory increment
tests/test_memory.py # Cosmos store logic (fake container; no live Cosmos needed)
```

Series

Prev: Week 2 ·

Next: Week 5 - MCP & Tools ·

All labs

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use:

agentic-ai-azure-week03-04-agent-framework.pptx

Prefer PDF? Download the **handout (slides + speaker notes)**:

agentic-ai-azure-week03-04-agent-framework-handout.pdf

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» [service.py](#)

```
"""Weeks 3-4 - Agent Framework: Wealth Management Research Assistant.
Demonstrates the Agent Framework concepts on top of a runnable service:
```

```

* **Memory** - per-session conversation memory (in-memory mock; Cosmos DB in prod).
* **Middleware** - input PII redaction + output compliance-disclaimer injection.
* **Grounding tools** - holdings + market-data lookups the agent uses as context.
Two backends: deterministic ``MockResearchBackend`` (offline, tested) and
``FoundryResearchBackend`` (azure-ai-projects v2 Responses API, lazy-imported).
"""
from __future__ import annotations
import re
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    foundry_project_endpoint: str = ""
    foundry_model_name: str = "gpt-4o"
    cosmos_connection_string: str = "" # set to enable durable Cosmos DB memory
    cosmos_database: str = "agentmemory"
    cosmos_container: str = "sessions"
    @property
    def use_foundry(self) -> bool:
        return bool(self.foundry_project_endpoint)
    @property
    def use_cosmos(self) -> bool:
        return bool(self.cosmos_connection_string)
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class ResearchRequest(BaseModel):
    advisor_query: str = Field(..., min_length=1, description="The advisor's natural-language request.")
    client_id: str = Field(default="HENDERSON", description="Portfolio/client identifier.")
    session_id: str = Field(default="default", description="Conversation session for memory.")
class ResearchResponse(BaseModel):
    answer: str
    sources: list[str]
    session_id: str
    memory_turns: int
    disclaimer: str
    redacted_input: bool
    mode: str
# grounding tools (mock data sources)
_HOLDINGS = {
    "HENDERSON": [
        {"symbol": "MSFT", "shares": 1200, "weight": 0.34},
        {"symbol": "AAPL", "shares": 800, "weight": 0.21},
        {"symbol": "VTI", "shares": 500, "weight": 0.45},
    ]
}
_MARKET = {"MSFT": 432.1, "AAPL": 214.5, "VTI": 268.9, "trend": "moderately bullish"}
def get_holdings(client_id: str) -> list[dict]:
    return _HOLDINGS.get(client_id.upper(), [])
def get_market_data() -> dict:
    return _MARKET
# middleware
_PII_PATTERNS = [
    re.compile(r"[\w.-]+@[ \w-]+\.[\w.-]+"), # email
    re.compile(r"\b\d{3}-\d{2}-\d{4}\b"), # SSN-like
    re.compile(r"\b(?:\+?\d{8,})\b"), # phone
]
DISCLAIMER = (
    "This is general information, not personalized financial advice. "
    "Past performance does not guarantee future results."
)
def redact_pii(text: str) -> tuple[str, bool]:
    redacted = text
    for pat in _PII_PATTERNS:
        redacted = pat.sub("[REDACTED]", redacted)
    return redacted, redacted != text
# session memory: in-memory (mock) or durable Cosmos DB
class InMemoryStore:
    """Process-local memory - fine for dev/tests, lost on restart."""
    backend = "in-memory"
    def __init__(self) -> None:
        self._data: dict[str, list[dict]] = {}
    def append(self, session_id: str, query: str, answer: str) -> int:
        turns = self._data.setdefault(session_id, [])
        turns.append({"query": query, "answer": answer})

```

```

        return len(turns)
    def recent(self, session_id: str, n: int = 2) -> list[dict]:
        return self._data.get(session_id, [])[-n:]

class CosmosMemoryStore:
    """Durable memory in Azure Cosmos DB, partitioned by `sessionId`.
    Lazy-imports the SDK and creates the database/container on first use.
    """
    backend = "cosmos"
    def __init__(self) -> None:
        from azure.cosmos import CosmosClient, PartitionKey
        s = get_settings()
        client = CosmosClient.from_connection_string(s.cosmos_connection_string)
        db = client.create_database_if_not_exists(id=s.cosmos_database)
        self._container = db.create_container_if_not_exists(
            id=s.cosmos_container, partition_key=PartitionKey(path="/sessionId")
        )
    def append(self, session_id: str, query: str, answer: str) -> int:
        import time
        import uuid
        self._container.upsert_item(
            {
                "id": uuid.uuid4().hex,
                "sessionId": session_id,
                "query": query,
                "answer": answer,
                "ts": time.time_ns(), # high-resolution, strictly increasing per call
            }
        )
        rows = list(
            self._container.query_items(
                query="SELECT VALUE COUNT(1) FROM c WHERE c.sessionId = @s",
                parameters=[{"name": "@s", "value": session_id}],
                partition_key=session_id,
            )
        )
        return rows[0] if rows else 0
    def recent(self, session_id: str, n: int = 2) -> list[dict]:
        items = list(
            self._container.query_items(
                query="SELECT * FROM c WHERE c.sessionId = @s ORDER BY c.ts DESC",
                parameters=[{"name": "@s", "value": session_id}],
                partition_key=session_id,
            )
        )
        return list(reversed(items[:n]))

@lru_cache
def get_memory_store():
    return CosmosMemoryStore() if get_settings().use_cosmos else InMemoryStore()
def _recent_context(session_id: str) -> str:
    turns = get_memory_store().recent(session_id, n=2)
    if not turns:
        return ""
    return " | ".join(f"Q:{t['query']}" for t in turns)
SYSTEM_INSTRUCTIONS = (
    "You are a wealth-management research assistant for financial advisors. Use the provided "
    "holdings and market data to answer concisely and professionally. Do not invent figures."
)
# backends
class MockResearchBackend:
    mode = "mock"
    def research(self, req: ResearchRequest) -> ResearchResponse:
        clean_query, redacted = redact_pii(req.advisor_query)
        holdings = get_holdings(req.client_id)
        market = get_market_data()
        if holdings:
            lines = [f"{h['symbol']} {h['shares']} sh @ ${market.get(h['symbol'], 0)}" for h in holdings]
            body = (
                f"Portfolio '{req.client_id}' holds: {'', '.join(lines)}. "
                f"Market is {market['trend']}. Regarding '{clean_query.strip()}': the allocation looks "
                f"diversified across equities and a broad-market ETF."
            )
        else:
            body = f"No holdings found for client '{req.client_id}'."
        answer = f"{body}\n\n{DISCLAIMER}"
        turns = get_memory_store().append(req.session_id, clean_query, body)
        return ResearchResponse(
            answer=answer,
            sources=["tool:get_holdings", "tool:get_market_data"],

```

```

        session_id=req.session_id,
        memory_turns=turns,
        disclaimer=DISCLAIMER,
        redacted_input=redacted,
        mode=self.mode,
    )
class FoundryResearchBackend:
    mode = "foundry"
    def research(self, req: ResearchRequest) -> ResearchResponse:
        from azure.ai.projects import AIProjectClient
        from azure.identity import DefaultAzureCredential
        clean_query, redacted = redact_pii(req.advisor_query)
        holdings = get_holdings(req.client_id)
        market = get_market_data()
        context = (
            f"Holdings({req.client_id})={holdings}\nMarket={market}\n"
            f"RecentContext={_recent_context(req.session_id)}"
        )
        s = get_settings()
        with (
            DefaultAzureCredential() as cred,
            AIProjectClient(endpoint=s.foundry_project_endpoint, credential=cred) as proj,
        ):
            client = proj.get_openai_client()
            resp = client.responses.create(
                model=s.foundry_model_name,
                input=[
                    {"role": "system", "content": SYSTEM_INSTRUCTIONS},
                    {"role": "user", "content": f"{context}\n\nAdvisor asks: {clean_query}"},
                ],
            )
            body = resp.output_text or "No response."
            answer = f"{body}\n\n{DISCLAIMER}" # middleware: disclaimer injection
            turns = get_memory_store().append(req.session_id, clean_query, body)
            return ResearchResponse(
                answer=answer,
                sources=["tool:get_holdings", "tool:get_market_data", "foundry:responses"],
                session_id=req.session_id,
                memory_turns=turns,
                disclaimer=DISCLAIMER,
                redacted_input=redacted,
                mode=self.mode,
            )
def get_backend():
    return FoundryResearchBackend() if get_settings().use_foundry else MockResearchBackend()

```

» main.py

```

"""Weeks 3-4 - Microsoft Agent Framework.
Wealth Management Research Assistant with memory + middleware + grounding tools.
Runs in MOCK mode out of the box. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.service import ResearchRequest, ResearchResponse, get_backend, get_settings
settings = get_settings()
app = FastAPI(title="Weeks 3-4 - Agent Framework (Wealth Research)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "week": "3-4",
        "backend": "foundry" if settings.use_foundry else "mock",
        "memory": "cosmos" if settings.use_cosmos else "in-memory",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week03-04-agent-framework",
        "endpoint": "/api/v1/research",
        "backend": "foundry" if settings.use_foundry else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/research", response_model=ResearchResponse, tags=["week03-04"])
def research(payload: ResearchRequest) -> ResearchResponse:
    return get_backend().research(payload)

```

Week 5 - Tools, MCP & Interoperability

Week 5 - Tools, MCP & Interoperability

> **Run in VS Code - no Azure needed.** pip install -r requirements.txt, then uvicorn app.main:app --reload and open <http://127.0.0.1:8000/docs>. Runs in **mock mode** by default - no az login, keys, or .env required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Connect an agent to tools via standardized contracts (REST/OpenAPI/Functions/MCP) so tools are reusable.

Enterprise use case - "Procurement Operations Agent" (Manufacturing / Retail)

The agent checks **inventory**, gets **supplier pricing**, and creates a **purchase order** - each a tool with a JSON-schema contract, with **idempotency** on the write tool.

[x] What this repo implements

- A **tool registry** (check_inventory, get_supplier_pricing, create_purchase_order) with JSON-schema definitions - the contract shared by MCP and OpenAI function-calling.
- **MCP-style discovery** at GET /api/v1/tools.
- **Idempotency** guard so retrying a PO with the same idempotency_key returns the same order.
- **Mock backend** (offline, tested) and **Foundry backend** (Responses API tool-calling loop).

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl http://127.0.0.1:8000/api/v1/tools
curl -X POST http://127.0.0.1:8000/api/v1/procure \
-H "Content-Type: application/json" \
-d '{"request": "Order 500 units of SKU-9981", "idempotency_key": "po-001"}'
```

Run tests: pytest -q. Known SKUs: SKU-9981, SKU-1001, SKU-5500.

Foundry mode

az login, then set FOUNDRY_PROJECT_ENDPOINT + FOUNDRY_MODEL_NAME. The model calls the registry tools during the run; the result is re-validated deterministically.

Architect's lens

- **MCP as the interoperability contract** - decouple tools from agents so they're portable.
- Tool governance: schemas as the trust boundary, **idempotency** for write tools.
- Gateway pattern: centralize auth/rate-limits/observability at **API Management**.

Tech stack

MCP, OpenAPI, Azure Functions, Logic Apps, API Management, FastAPI, azure-ai-projects v2.

Structure

```

app/service.py # settings, schemas, tool registry + funcs, backends
app/main.py   # POST /api/v1/procure, GET /api/v1/tools
tests/test_app.py

```

Series

Prev: Weeks 3-4 ·

Next: Weeks 6-7 - Multi-agent ·

All labs

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-week05-mcp-tools.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-week05-mcp-tools-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `service.py`

```

"""Week 5 - Tools, MCP & Interoperability: Procurement Operations Agent.
Demonstrates tool contracts and interoperability: a small **tool registry**
(inventory check, supplier pricing, PO creation) with JSON-schema definitions, an
**idempotency** guard on the write tool, and an MCP-style **tool discovery** endpoint.
Two backends: ``MockProcurementBackend`` (offline, tested) and
``FoundryProcurementBackend`` (azure-ai-projects v2 Responses API with the tools
registered as function tools, lazy-imported).
"""

from __future__ import annotations
import json
import re
import uuid
from functools import lru_cache
from typing import Any
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict

# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    foundry_project_endpoint: str = ""
    foundry_model_name: str = "gpt-4o"
    @property
    def use_foundry(self) -> bool:
        return bool(self.foundry_project_endpoint)
    @lru_cache
    def get_settings() -> Settings:
        return Settings()

# schemas
class ProcureRequest(BaseModel):
    request: str = Field(..., min_length=1, description="Procurement request in natural language.")
    idempotency_key: str | None = Field(default=None, description="Dedupe key for PO creation.")

class ProcureResponse(BaseModel):
    order_id: str | None
    status: str
    sku: str | None
    quantity: int | None
    unit_price: float | None
    total: float | None
    steps: list[str]
    mode: str

# tools (with JSON schemas for MCP / function-calling)
_INVENTORY = {"SKU-9981": 1200, "SKU-1001": 0, "SKU-5500": 350}
_PRICING = {"SKU-9981": 4.25, "SKU-1001": 19.99, "SKU-5500": 8.10}
_PO_STORE: dict[str, dict] = {} # idempotency_key -> order
def check_inventory(sku: str) -> dict[str, Any]:
    qty = _INVENTORY.get(sku.upper())
    return {"sku": sku, "known": qty is not None, "in_stock": (qty or 0) > 0, "available": qty or 0}

```

```

def get_supplier_pricing(sku: str) -> dict[str, Any]:
    price = _PRICING.get(sku.upper())
    return {"sku": sku, "unit_price": price, "currency": "USD", "found": price is not None}
def create_purchase_order(sku: str, quantity: int, idempotency_key: str | None = None) -> dict[str, Any]:
    if idempotency_key and idempotency_key in _PO_STORE:
        return {**_PO_STORE[idempotency_key], "idempotent_replay": True}
    order = {"order_id": f"PO-{uuid.uuid4().hex[:8].upper()}", "sku": sku, "quantity": quantity, "status": "created"}
    if idempotency_key:
        _PO_STORE[idempotency_key] = order
    return {**order, "idempotent_replay": False}
TOOL_REGISTRY: list[dict[str, Any]] = [
    {
        "type": "function",
        "name": "check_inventory",
        "description": "Check current inventory level for a SKU.",
        "parameters": {
            "type": "object",
            "properties": {"sku": {"type": "string"}},
            "required": ["sku"],
            "additionalProperties": False,
        },
    },
    {
        "type": "function",
        "name": "get_supplier_pricing",
        "description": "Get the supplier unit price for a SKU.",
        "parameters": {
            "type": "object",
            "properties": {"sku": {"type": "string"}},
            "required": ["sku"],
            "additionalProperties": False,
        },
    },
    {
        "type": "function",
        "name": "create_purchase_order",
        "description": "Create a purchase order for a SKU and quantity (idempotent).",
        "parameters": {
            "type": "object",
            "properties": {
                "sku": {"type": "string"},
                "quantity": {"type": "integer"},
                "idempotency_key": {"type": "string"},
            },
            "required": ["sku", "quantity"],
            "additionalProperties": False,
        },
    },
]
_TOOL_FUNCS = {
    "check_inventory": check_inventory,
    "get_supplier_pricing": get_supplier_pricing,
    "create_purchase_order": create_purchase_order,
}
_SKU_RE = re.compile(r"\bSKU-\d{3,6}\b", re.IGNORECASE)
_QTY_RE = re.compile(r"\b\d{1,6}\b")
SYSTEM_INSTRUCTIONS = (
    "You are a procurement operations agent. Given a request, determine the SKU and quantity, "
    "check inventory, get supplier pricing, and create a purchase order using the tools. "
    "Always check inventory and pricing before creating a PO."
)
# backends
class MockProcurementBackend:
    mode = "mock"
    def procure(self, req: ProcureRequest) -> ProcureResponse:
        steps: list[str] = []
        sku_m = _SKU_RE.search(req.request)
        sku = sku_m.group(0).upper() if sku_m else None
        qty_m = _QTY_RE.search(req.request)
        qty = int(qty_m.group(1)) if qty_m else None
        steps.append(f"Parsed sku={sku}, quantity={qty}.")
        if not sku or not qty:
            return ProcureResponse(
                order_id=None, status="needs_clarification", sku=sku, quantity=qty,
                unit_price=None, total=None, mode=self.mode,
                steps=steps + ["Could not determine SKU and/or quantity."],
            )
        inv = check_inventory(sku)

```

```

steps.append(f"check_inventory -&gt; available={inv['available']}.")
pricing = get_supplier_pricing(sku)
steps.append(f"get_supplier_pricing -&gt; unit_price={pricing['unit_price']}.")
if not pricing["found"]:
    return ProcureResponse(
        order_id=None, status="rejected", sku=sku, quantity=qty,
        unit_price=None, total=None, mode=self.mode,
        steps=steps + ["No supplier price; cannot order."],
    )
order = create_purchase_order(sku, qty, req.idempotency_key)
steps.append(f"create_purchase_order -&gt; {order['order_id']} (replay={order['idempotent_replay']}).")
total = round(pricing["unit_price"] * qty, 2)
return ProcureResponse(
    order_id=order["order_id"], status=order["status"], sku=sku, quantity=qty,
    unit_price=pricing["unit_price"], total=total, steps=steps, mode=self.mode,
)
class FoundryProcurementBackend:
    mode = "foundry"
    def procure(self, req: ProcureRequest) -&gt; ProcureResponse:
        from azure.ai.projects import AIProjectClient
        from azure.identity import DefaultAzureCredential
        steps: list[str] = []
        s = get_settings()
        with (
            DefaultAzureCredential() as cred,
            AIProjectClient(endpoint=s.foundry_project_endpoint, credential=cred) as proj,
        ):
            client = proj.get_openai_client()
            response = client.responses.create(
                model=s.foundry_model_name,
                input=[
                    {"role": "system", "content": SYSTEM_INSTRUCTIONS},
                    {"role": "user", "content": req.request},
                ],
                tools=TOOL_REGISTRY,
            )
            for _ in range(6):
                calls = [it for it in response.output if getattr(it, "type", None) == "function_call"]
                if not calls:
                    break
                outputs = []
                for call in calls:
                    args = json.loads(call.arguments or "{}")
                    if call.name == "create_purchase_order" and req.idempotency_key:
                        args.setdefault("idempotency_key", req.idempotency_key)
                    result = _TOOL_FUNCS[call.name](**args)
                    steps.append(f"Model called {call.name} -&gt; {result}.")
                    outputs.append(
                        {"type": "function_call_output", "call_id": call.call_id, "output": json.dumps(result)}
                    )
                response = client.responses.create(
                    model=s.foundry_model_name, previous_response_id=response.id,
                    input=outputs, tools=TOOL_REGISTRY,
                )
            # Re-derive a structured summary from the mock store / tools deterministically.
            mock = MockProcurementBackend().procure(req)
            mock.steps = steps + ["Finalized via deterministic re-validation."]
            mock.mode = self.mode
            return mock
    def get_backend():
        return FoundryProcurementBackend() if get_settings().use_foundry else MockProcurementBackend()

```

» main.py

```

"""Week 5 - Tools, MCP & Interoperability.
Procurement Operations Agent with a tool registry, idempotency, and MCP-style
tool discovery. Runs in MOCK mode out of the box. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.service import (
    TOOL_REGISTRY,
    ProcureRequest,
    ProcureResponse,
    get_backend,
    get_settings,
)

```

```

settings = get_settings()
app = FastAPI(title="Week 5 - MCP & Tools (Procurement)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {"status": "ok", "week": "5", "backend": "foundry" if settings.use_foundry else "mock"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week05-mcp-tools",
        "endpoint": "/api/v1/procure",
        "backend": "foundry" if settings.use_foundry else "mock",
        "docs": "/docs",
    }
@app.get("/api/v1/tools", tags=["week05"])
def list_tools() -> dict[str, list]:
    """MCP-style tool discovery: advertise the callable tool contracts."""
    return {"tools": [{"name": t["name"], "description": t["description"]} for t in TOOL_REGISTRY]}
@app.post("/api/v1/procure", response_model=ProcureResponse, tags=["week05"])
def procure(payload: ProcureRequest) -> ProcureResponse:
    return get_backend().procure(payload)

```

Weeks 6-7 - Multi-Agent Orchestration

Weeks 6-7 - Multi-Agent Orchestration Patterns

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no az login, keys, or .env required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Implement sequential, **concurrent**, handoff, and **human-in-the-loop** orchestration.

Enterprise use case - "Loan Underwriting Pipeline" (Banking)

Intake -> (Credit-Risk || Compliance, concurrent) -> Underwriter handoff, with human approval for edge cases.

[x] What this repo implements

- **Sequential + concurrent** agents - intake first, then credit-risk and compliance via `asyncio.gather`.
- **Handoff** to an underwriter agent that applies decision rules.
- **Human-in-the-loop** - large/medium-risk loans pause as `awaiting_approval`; resume via `approve` endpoint.
- **Run store + status polling** - every run is addressable; Cosmos DB is the prod backing store.

Fully runnable offline; no Azure required. (FOUNDRY_PROJECT_ENDPOINT only flips the reported mode.)

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
# Large loan -> pauses for human approval
curl -X POST http://127.0.0.1:8000/api/v1/underwrite \
  -H "Content-Type: application/json" \
  -d '{"applicant_summary": "Applicant: J. Smith, income $95k, requesting $300k mortgage, credit score 712."}'
# -> note the run_id, then:
curl -X POST http://127.0.0.1:8000/api/v1/runs/{run_id}/approve \
  -H "Content-Type: application/json" -d '{"approved": true}'
```

Run tests: `pytest -q`

Endpoints

Method	Path	Purpose
POST	<code>/api/v1/underwrite</code>	Start the pipeline
GET	<code>/api/v1/runs/{run_id}</code>	Poll run status
POST	<code>/api/v1/runs/{run_id}/approve</code>	Human-in-the-loop resume

Architect's lens

- Concurrency (independent sub-tasks) vs. sequential dependencies.
- Durable state for long-running approvals; **Durable Functions / Durable Tasks** in prod.
- Inter-agent comms via **A2A** + event-driven topologies (Service Bus / Event Grid).
- Failure handling: partial results, compensation, retries.

Tech stack

Agent Framework orchestration, A2A, Service Bus / Event Grid, Durable Functions, FastAPI, Cosmos DB.

Structure

```
app/service.py    # agents, orchestration (asyncio.gather), run store, HITL resume
app/main.py       # underwrite + status + approve endpoints
tests/test_app.py
```

Series

Prev: Week 5 ·

Next: Week 8 - RAG ·

All labs

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-week06-07-multi-agent.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-week06-07-multi-agent-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `service.py`

```
"""Weeks 6-7 - Multi-Agent Orchestration: Loan Underwriting Pipeline.
Demonstrates multi-agent patterns on a runnable service:
* **Sequential + concurrent** - an intake agent runs first, then the credit-risk
  and compliance agents run **concurrently** (`asyncio.gather`).
* **Handoff** - results hand off to the underwriter agent for a decision.
* **Human-in-the-loop** - edge cases pause as `awaiting_approval` and are resumed
  via an approve endpoint; run state lives in a store (Cosmos DB in prod).
The pipeline is deterministic and fully offline/testable. `FOUNDRY_PROJECT_ENDPOINT`
toggles a `mode` flag; the orchestration itself is the lesson.
"""

from __future__ import annotations
import asyncio
import re
import uuid
import uuid
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    foundry_project_endpoint: str = ""
    foundry_model_name: str = "gpt-4o"
    hitl_amount_threshold: float = 250_000.0
    @property
    def use_foundry(self) -> bool:
        return bool(self.foundry_project_endpoint)
    @lru_cache
    def get_settings() -> Settings:
        return Settings()
# schemas
class UnderwriteRequest(BaseModel):
    applicant_summary: str = Field(..., min_length=1, description="Free-text loan application summary.")
class ApproveRequest(BaseModel):
    approved: bool
    note: str | None = None
class UnderwriteRun(BaseModel):
    run_id: str
    status: str # running | awaiting_approval | completed
    decision: str # pending | approved | rejected
```

```

    applicant: dict
    risk: dict
    compliance: dict
    steps: list[str]
    mode: str
# run store (Cosmos DB in prod)
_RUNS: dict[str, UnderwriteRun] = {}
def get_run(run_id: str) -> UnderwriteRun | None:
    return _RUNS.get(run_id)
# agents
def intake_agent(text: str) -> dict:
    name_m = re.search(r"applicant[:\s]+([A-Za-z.\- ]+)?(?:,|$)", text, re.IGNORECASE)
    score_m = re.search(r"(?:credit score|score)[^\d]*(\d{3})", text, re.IGNORECASE)
    income = _money(re.search(r"income[^\d]*\${?}\s*([\d,]+\s*(k)?", text, re.IGNORECASE))
    amount = _money(
        re.search(r"(?:requesting|loan|mortgage|amount|borrow)[^\d]*\${?}\s*([\d,]+\s*(k)?", text, re
.IGNORECASE)
    )
    return {
        "name": name_m.group(1).strip() if name_m else None,
        "credit_score": int(score_m.group(1)) if score_m else None,
        "income": income,
        "amount": amount,
    }
async def credit_risk_agent(applicant: dict) -> dict:
    score = applicant.get("credit_score") or 0
    if score >= 680:
        level = "low"
    elif score >= 600:
        level = "medium"
    else:
        level = "high"
    return {"risk_level": level, "credit_score": score}
async def compliance_agent(applicant: dict, text: str) -> dict:
    flagged = any(w in text.lower() for w in ("sanction", "watchlist", "fraud"))
    return {"kyc": "pass", "aml": "flagged" if flagged else "pass", "passed": not flagged}
def underwriter_agent(applicant: dict, risk: dict, compliance: dict, threshold: float) -> tuple[str,
str]:
    """Returns (status, decision)."""
    if not compliance["passed"]:
        return "completed", "rejected"
    if risk["risk_level"] == "high":
        return "completed", "rejected"
    amount = applicant.get("amount") or 0
    if risk["risk_level"] == "medium" or amount > threshold:
        return "awaiting_approval", "pending" # human-in-the-loop
    return "completed", "approved"
def _money(m: re.Match | None) -> float | None:
    if not m:
        return None
    val = float(m.group(1).replace(",", ""))
    if m.lastindex and m.lastindex >= 2 and (m.group(2) or "").lower() == "k":
        val *= 1000
    return val
# orchestration
async def run_underwriting(request: UnderwriteRequest) -> UnderwriteRun:
    settings = get_settings()
    steps = ["Intake agent extracting application data."]
    applicant = intake_agent(request.applicant_summary)
    steps.append("Running credit-risk + compliance agents concurrently.")
    risk, compliance = await asyncio.gather(
        credit_risk_agent(applicant),
        compliance_agent(applicant, request.applicant_summary),
    )
    steps.append(f"Risk={risk['risk_level']}, compliance_passed={compliance['passed']}.")
    status, decision = underwriter_agent(applicant, risk, compliance, settings.hitl_amount_threshold)
)
    steps.append(
        "Underwriter decision: human approval required."
        if status == "awaiting_approval"
        else f"Underwriter decision: {decision}."
    )
    run = UnderwriteRun(
        run_id=f"RUN-{uuid.uuid4().hex[:8].upper()}",
        status=status,
        decision=decision,
        applicant=applicant,
        risk=risk,
        compliance=compliance,
        steps=steps,

```

```

        mode="foundry" if settings.use_foundry else "mock",
    )
    _RUNS[run.run_id] = run
    return run
def resume_underwriting(run_id: str, approval: ApproveRequest) -> UnderwriteRun | None:
    run = _RUNS.get(run_id)
    if run is None or run.status != "awaiting_approval":
        return run
    run.decision = "approved" if approval.approved else "rejected"
    run.status = "completed"
    run.steps.append(f"Human approver resolved: {run.decision}" + (f" ({approval.note})" if approval
.note else "."))
    return run

```

» main.py

```

"""Weeks 6-7 - Multi-Agent Orchestration Patterns.
Loan Underwriting Pipeline: sequential intake -> concurrent credit-risk + compliance
-> underwriter handoff, with human-in-the-loop approval. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI, HTTPException
from app.service import (
    ApproveRequest,
    UnderwriteRequest,
    UnderwriteRun,
    get_run,
    get_settings,
    resume_underwriting,
    run_underwriting,
)
settings = get_settings()
app = FastAPI(title="Weeks 6-7 - Multi-Agent (Loan Underwriting)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {"status": "ok", "week": "6-7", "backend": "foundry" if settings.use_foundry else "mock"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week06-07-multi-agent",
        "endpoint": "/api/v1/underwrite",
        "backend": "foundry" if settings.use_foundry else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/underwrite", response_model=UnderwriteRun, tags=["week06-07"])
async def underwrite(payload: UnderwriteRequest) -> UnderwriteRun:
    """Start the underwriting pipeline. May complete or pause for human approval."""
    return await run_underwriting(payload)
@app.get("/api/v1/runs/{run_id}", response_model=UnderwriteRun, tags=["week06-07"])
def run_status(run_id: str) -> UnderwriteRun:
    run = get_run(run_id)
    if run is None:
        raise HTTPException(status_code=404, detail="Run not found.")
    return run
@app.post("/api/v1/runs/{run_id}/approve", response_model=UnderwriteRun, tags=["week06-07"])
def approve(run_id: str, payload: ApproveRequest) -> UnderwriteRun:
    """Human-in-the-loop resume for runs that are awaiting_approval."""
    run = resume_underwriting(run_id, payload)
    if run is None:
        raise HTTPException(status_code=404, detail="Run not found.")
    return run

```

Week 8 - Memory, State & Grounding (RAG)

Week 8 - Memory, State & Grounding (RAG)

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no `az login`, keys, or `.env` required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Build a retrieval-grounded agent with **permission-aware** knowledge over enterprise data.

Enterprise use case - "Clinical Policy Assistant" (Healthcare)

Clinicians query care protocols. The agent retrieves from a knowledge base, **trims** results by the user's role/department (security trimming), cites sources, and never answers beyond grounded content.

[x] What this repo implements

- **Retrieval** with relevance scoring (mock hybrid; **Azure AI Search** vector+hybrid+semantic in prod).
- **Security trimming** - each document carries a role/department ACL; unauthorized docs are dropped **before** reaching the model (the mock counts them as `trimmed_count`; the search backend pushes the ACL into an **OData filter**).
- **Citations** on every grounded answer + an **"I don't know"** guardrail for ungrounded questions.
- **Mock backend** (offline, tested) and **Search backend** (Azure AI Search + Foundry, lazy-imported).

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
# Authorized + grounded
curl -X POST http://127.0.0.1:8000/api/v1/policy/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "post-operative antibiotic protocol for hip replacement", "user_role": "clinician", "department": "orthopedics"}'
# Same question as pharmacist/pharmacy -> trimmed -> "I don't know"
```

Run tests: `pytest -q`

Wire the real Azure backend (Azure AI Search)

This lab ships a **real, runnable** Azure AI Search backend plus a one-time setup script that creates the index and ingests the sample clinical docs with embeddings.

1. Provision (Azure CLI)

[bash]

```
az login
RG=rg-agentic-rag
az group create -n $RG -l eastus
# Azure AI Search service
az search service create -g $RG -n my-clinical-search --sku basic
az search admin-key show -g $RG --service-name my-clinical-search # copy the primary key
```

You also need a **Microsoft Foundry** project with two deployments: a chat model (e.g. `gpt-4o`) and an **embedding** model (e.g. `text-embedding-3-small`, 1536 dims). Grant your identity the **Azure AI User** role on the Foundry project (`az login` provides the credential).

2. Configure .env

[bash]

```
cp .env.example .env

AZURE_AI_SEARCH_ENDPOINT=https://my-clinical-search.search.windows.net
AZURE_AI_SEARCH_KEY=<primary-admin-key>;
AZURE_AI_SEARCH_INDEX=clinical-policies
FOUNDRY_PROJECT_ENDPOINT=https://<account>.services.ai.azure.com/api/projects/<project>;
FOUNDRY_MODEL_NAME=gpt-4o
FOUNDRY_EMBEDDING_DEPLOYMENT=text-embedding-3-small
EMBEDDING_DIMENSIONS=1536
```

3. Create the index + ingest docs (one time)

[bash]

```
python scripts/setup_search.py
```

This creates a vector-enabled index with **ACL collection fields** (roles, departments), an **HNSW** vector profile, and a **semantic** config, then embeds and uploads the docs.

4. Run - now backed by Azure AI Search

[bash]

```
uvicorn app.main:app --reload # GET /health -> "backend": "search"
```

The query path (app/service.py -> SearchRagBackend) embeds the question, runs a **hybrid** query (keyword + vector, RRF-fused), pushes **security trimming** into an OData `roles/any(...)` and departments/any(...)` filter, and grounds the answer on the retrieved citations via Foundry. If no embedding deployment is set, it degrades to keyword-only search (still trimmed and grounded).

Without FOUNDRY_EMBEDDING_DEPLOYMENT you get keyword-only search; with it you get true hybrid.

Architect's lens

- Retrieval quality: chunking, hybrid search, reranking, recency.
- **Security trimming as a first-class requirement** (PHI/PII never crosses boundaries).
- Grounding vs. hallucination guardrails; explicit "I don't know".

Tech stack

Azure AI Search (vector/hybrid/semantic), Azure OpenAI embeddings, Blob Storage, Entra ID groups for ACL,
FastAPI, azure-ai-projects v2.

Structure

```
app/service.py          # settings, schemas, mock + real (hybrid) Search backends
app/main.py             # POST /api/v1/policy/ask
scripts/setup_search.py # one-time: create index + ingest docs with embeddings
tests/test_app.py
```

Series

Prev: Weeks 6-7 ·

Next: Week 9 - Hosting & Scale ·

All labs

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-week08-rag-grounding.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-week08-rag-grounding-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» service.py

```
"""Week 8 - Memory, State & Grounding (RAG): Clinical Policy Assistant.
Demonstrates retrieval-grounded answering with **permission-aware** knowledge:
* **Retrieval** - score documents against the question (mock hybrid scoring; Azure
  AI Search vector+hybrid+semantic in prod).
* **Security trimming** - drop documents the user's role/department may not see
  *before* they reach the model (a first-class requirement for PHI/PII).
* **Citations + grounding guardrail** - answers cite sources and say "I don't know"
  when nothing authorized is relevant.
`MockRagBackend` runs offline/tested; `SearchRagBackend` uses Azure AI Search +
Foundry, lazy-imported, when search settings are present.
"""

from __future__ import annotations
import re
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    foundry_project_endpoint: str = ""
    foundry_model_name: str = "gpt-4o"
    foundry_embedding_deployment: str = "text-embedding-3-small"
    embedding_dimensions: int = 1536
    azure_ai_search_endpoint: str = ""
    azure_ai_search_key: str = ""
    azure_ai_search_index: str = "clinical-policies"
    @property
    def use_search(self) -> bool:
        return bool(self.azure_ai_search_endpoint and self.azure_ai_search_key)
    @property
    def use_embeddings(self) -> bool:
        return bool(self.foundry_project_endpoint and self.foundry_embedding_deployment)
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class PolicyQuestion(BaseModel):
    question: str = Field(..., min_length=1, description="Clinician's question about a care protocol.")
    user_role: str = Field(default="clinician", description="Caller role for security trimming.")
    department: str = Field(default="general", description="Caller department for security trimming.")
class Citation(BaseModel):
    title: str
    snippet: str
    score: float
class PolicyAnswer(BaseModel):
    answer: str
    grounded: bool
    citations: list[Citation]
    trimmed_count: int = Field(..., description="Docs excluded by security trimming (ACL).")
    mode: str
# knowledge base (with per-doc ACLs) - shared by the mock backend and the
# setup_search.py ingestion script so both stay in sync.
CLINICAL_DOCS = [
    {
        "id": "doc-1",
        "title": "Hip Replacement Post-Op Protocol",
        "content": "Post-operative antibiotic protocol for hip replacement: cefazolin 2g IV "
        "every 8 hours for 24 hours. Monitor for surgical-site infection.",
        "roles": ["clinician"],
        "departments": ["orthopedics", "general"],
    },
    {
        "id": "doc-2",
        "title": "Sepsis Care Pathway",
        "content": "Sepsis pathway: measure serum lactate, obtain blood cultures, and start "
        "broad-spectrum antibiotics within one hour of recognition.",
        "roles": ["clinician"],
        "departments": ["general", "icu"],
    },
]
```

```

{
    "id": "doc-3",
    "title": "Controlled Substance Handling (Pharmacy)",
    "content": "Controlled substances require dual sign-off and reconciliation each shift.",
    "roles": ["pharmacist"],
    "departments": ["pharmacy"],
},
]
_DOCS = CLINICAL_DOCS # backwards-compatible alias used by the mock retriever
_WORD_RE = re.compile(r"[a-z]{3,}")
def _tokens(text: str) -> set[str]:
    return set(_WORD_RE.findall(text.lower()))
def _visible(doc: dict, role: str, department: str) -> bool:
    """Security trimming: caller must satisfy the document's role + department ACL."""
    return role.lower() in doc["roles"] and department.lower() in doc["departments"]
def retrieve(question: str, role: str, department: str) -> tuple[list[Citation], int]:
    """Return (ranked citations, count_trimmed_by_acl)."""
    q = _tokens(question)
    trimmed = 0
    scored: list[Citation] = []
    for doc in _DOCS:
        if not _visible(doc, role, department):
            trimmed += 1
            continue
        overlap = q & _tokens(doc["title"] + " " + doc["content"])
        score = round(len(overlap) / (len(q) or 1), 3)
        if score > 0:
            scored.append(Citation(title=doc["title"], snippet=doc["content"][:160], score=score))
    scored.sort(key=lambda c: c.score, reverse=True)
    return scored, trimmed
SYSTEM_INSTRUCTIONS = (
    "You are a clinical policy assistant. Answer ONLY from the provided context. "
    "Cite the document titles you used. If the context does not contain the answer, say you don't know."
)
# backends
class MockRagBackend:
    mode = "mock"
    def ask(self, q: PolicyQuestion) -> PolicyAnswer:
        citations, trimmed = retrieve(q.question, q.user_role, q.department)
        if not citations:
            return PolicyAnswer(
                answer="I don't know based on the available, authorized documents.",
                grounded=False, citations=[], trimmed_count=trimmed, mode=self.mode,
            )
        top = citations[0]
        answer = f"{top.snippet} (Source: {top.title})"
        return PolicyAnswer(
            answer=answer, grounded=True, citations=citations[:3], trimmed_count=trimmed, mode=self.mode,
        )
#: name of the vector field in the index (must match setup_search.py)
VECTOR_FIELD = "contentVector"
class SearchRagBackend:
    """Real Azure AI Search backend: hybrid (keyword + vector) retrieval with
    security-trimming pushed into the query, then grounded generation on Foundry.
    """
    mode = "search"
    def ask(self, q: PolicyQuestion) -> PolicyAnswer:
        from azure.core.credentials import AzureKeyCredential
        from azure.search.documents import SearchClient
        s = get_settings()
        sc = SearchClient(
            endpoint=s.azure_ai_search_endpoint,
            index_name=s.azure_ai_search_index,
            credential=AzureKeyCredential(s.azure_ai_search_key),
        )
        # Security trimming: an OData filter on the ACL collection fields. Documents
        # the caller's role/department can't see never enter the result set.
       flt = (
            f"roles/any(r: r eq '{q.user_role}')" and "
            f"departments/any(d: d eq '{q.department}')"
        )
        # When Foundry embeddings are configured we run a *hybrid* query (keyword +
        # vector, fused with RRF) and generate the answer in the same connection.
        if s.use_embeddings:
            return self._hybrid_ask(q, sc, flt, s)
        # Keyword-only fallback (still security-trimmed and grounded).
        results = list(sc.search(search_text=q.question, filter=flt, top=3))
        citations = _to_citations(results)

```

```

        if not citations:
            return _idk(self.mode)
        answer = f"{citations[0].snippet} (Source: {citations[0].title})"
        return PolicyAnswer(answer=answer, grounded=True, citations=citations, trimmed_count=0, mode
= self.mode)
    def _hybrid_ask(self, q: PolicyQuestion, sc, flt: str, s) -> PolicyAnswer:
        from azure.ai.projects import AIProjectClient
        from azure.identity import DefaultAzureCredential
        from azure.search.documents.models import VectorizedQuery
        with (
            DefaultAzureCredential() as cred,
            AIProjectClient(endpoint=s.foundry_project_endpoint, credential=cred) as proj,
        ):
            oai = proj.get_openai_client()
            query_vector = oai.embeddings.create(
                model=s.foundry_embedding_deployment, input=q.question
            ).data[0].embedding
            vector_query = VectorizedQuery(
                vector=query_vector, k_nearest_neighbors=5, fields=VECTOR_FIELD
            )
            results = list(
                sc.search(
                    search_text=q.question,          # keyword leg
                    vector_queries=[vector_query],    # vector leg (RRF-fused)
                    filter=flt,                       # security trimming
                    select=["id", "title", "content", "roles", "departments"],
                    top=3,
                )
            )
            citations = _to_citations(results)
            if not citations:
                return _idk(self.mode)
            context = "\n".join(f"[{c.title}] {c.snippet}" for c in citations)
            resp = oai.responses.create(
                model=s.foundry_model_name,
                input=[
                    {"role": "system", "content": SYSTEM_INSTRUCTIONS},
                    {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {q.question}"},
                ],
            )
            answer = resp.output_text or f"{citations[0].snippet} (Source: {citations[0].title})"
            return PolicyAnswer(answer=answer, grounded=True, citations=citations, trimmed_count=0, mode
= self.mode)
    def _to_citations(results) -> list[Citation]:
        return [
            Citation(
                title=r.get("title", "doc"),
                snippet=(r.get("content", "") or "")[:160],
                score=float(r.get("@search.score", 0.0)),
            )
            for r in results
        ]
    def _idk(mode: str) -> PolicyAnswer:
        return PolicyAnswer(
            answer="I don't know based on the available, authorized documents.",
            grounded=False, citations=[], trimmed_count=0, mode=mode,
        )
    def get_backend():
        return SearchRagBackend() if get_settings().use_search else MockRagBackend()

```

» main.py

```

"""Week 8 - Memory, State & Grounding (RAG).
Clinical Policy Assistant with permission-aware retrieval, citations, and an
"I don't know" guardrail. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.service import PolicyAnswer, PolicyQuestion, get_backend, get_settings
settings = get_settings()
app = FastAPI(title="Week 8 - RAG (Clinical Policy Assistant)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {"status": "ok", "week": "8", "backend": "search" if settings.use_search else "mock"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week08-rag-grounding",
        "endpoint": "/api/v1/policy/ask",
        "backend": "search" if settings.use_search else "mock",
    }

```

```
        "docs": "/docs",
    }
@app.post("/api/v1/policy/ask", response_model=PolicyAnswer, tags=["week08"])
def ask(payload: PolicyQuestion) -> PolicyAnswer:
    return get_backend().ask(payload)
```

Week 9 - Hosting, Deployment & Scale

Week 9 - Hosting, Deployment & Scale

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no az login, keys, or .env required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Package and deploy agents for production scale, with **durable long-running tasks** and **status polling**.

Enterprise use case - "Black-Friday Customer Service Swarm" (Retail / E-commerce)

A fleet of CS agents scales from 10 to 10,000 sessions, scales to zero overnight, and runs durable **refund** tasks without blocking the request path.

[x] What this repo implements

- **Fast path** - FAQ-style messages answer inline.
- **Durable path** - refunds spawn a background task (FastAPI BackgroundTasks; ****Durable Functions / Durable Tasks**** in prod) with a pollable status at `GET /api/v1/tasks/{id}`.
- **Scale knobs** at `GET /api/v1/scale` (the signals KEDA uses).
- **Dockerfile + Gunicorn/Uvicorn** for the Container Apps deploy story.

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/support \
-H "Content-Type: application/json" \
-d '{"customer_message": "I want a refund for order #88231, it arrived damaged."}'
# -&gt; {"kind":"task","task_id":"TASK-...", "state":"processing"} then poll:
curl http://127.0.0.1:8000/api/v1/tasks/&lt;task_id&gt;
```

Run tests: `pytest -q`

Deploy (Container Apps)

[bash]

```
docker build -t cs-swarm .
docker run -p 8000:8000 cs-swarm # CMD uses gunicorn + uvicorn workers
```

Then push to ACR and deploy to **Azure Container Apps** with KEDA (scale-to-zero). Compare targets:

	Container Apps	AKS	Functions
Best for	most agent APIs	platform control	bursty/short
Scaling	KEDA, scale-to-zero	HPA/KEDA (you manage)	per-execution
Ops burden	low	high	lowest

Architect's lens

- Decision matrix above - **Container Apps is the course default**.

- Async FastAPI + Gunicorn/Uvicorn workers; connection pooling to models.
- Cost: token budgets, model routing (small for triage, large for hard cases), caching.

Tech stack

Docker, Azure Container Apps (+KEDA), AKS, Azure Functions, ACR, FastAPI + Gunicorn/Uvicorn, Bicep/Terraform.

Series

Prev: [Week 8](#) ·

Next: [Week 10 - Observability](#) ·

[All labs](#)

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: [agentic-ai-azure-week09-hosting-scale.pptx](#)

Prefer PDF? Download the **handout (slides + speaker notes)**:

[agentic-ai-azure-week09-hosting-scale-handout.pdf](#)

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» [service.py](#)

```
"""Week 9 - Hosting, Deployment & Scale: Customer Service Swarm.
Demonstrates the production runtime concerns:
* **Fast path vs. durable path** - FAQ-style messages answer inline; refunds are
  **long-running** and run as a background task with a pollable status (Durable
  Functions / Durable Tasks in prod).
* **Status polling** - every durable task is addressable by id.
* **Scale knobs** - concurrency settings exposed for the autoscaling story.
Fully runnable offline; the Dockerfile + Container Apps notes (README) cover deploy.
"""
from __future__ import annotations
import re
import uuid
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    max_concurrent_sessions: int = 1000
    worker_count: int = 4
    @property
    def use_foundry(self) -> bool:
        return False
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class SupportRequest(BaseModel):
    customer_message: str = Field(..., min_length=1, description="Incoming customer-service message.")
    order_id: str | None = None
class SupportResponse(BaseModel):
    kind: str # "answer" (fast path) | "task" (durable path)
    answer: str | None = None
    task_id: str | None = None
    state: str | None = None # processing | completed | failed
class TaskRecord(BaseModel):
    task_id: str
    state: str
    kind: str
    result: dict | None = None
```

```

    message: str
# durable task store (Durable Tasks / Cosmos in prod)
_TASKS: dict[str, TaskRecord] = {}
def get_task(task_id: str) -> TaskRecord | None:
    return _TASKS.get(task_id)
_ORDER_RE = re.compile(r"#{?b(\\d{4,8})\\b}")
_FAQ = {
    "hours": "Our support team is available 24/7 during the holiday season.",
    "track": "You can track your order from the 'My Orders' page using your order number.",
    "return": "Items can be returned within 30 days in original packaging.",
}
def answer_faq(message: str) -> str:
    low = message.lower()
    for key, val in _FAQ.items():
        if key in low:
            return val
    return "Thanks for reaching out - a customer-service agent will follow up shortly."
def process_refund(task_id: str, message: str, order_id: str | None) -> None:
    """The durable unit of work. In prod this is a Durable Functions activity."""
    rec = _TASKS[task_id]
    order = order_id
    if not order:
        m = _ORDER_RE.search(message)
        order = m.group(1) if m else None
    if not order:
        rec.state = "failed"
        rec.result = {"reason": "no_order_id"}
        return
    rec.state = "completed"
    rec.result = {"order_id": order, "status": "refunded", "amount_estimate": "full"}
def is_refund(message: str) -> bool:
    return any(w in message.lower() for w in ("refund", "money back", "chargeback"))
def start_refund_task(message: str, order_id: str | None) -> TaskRecord:
    task_id = f"TASK-{uuid.uuid4().hex[:8].upper()}"
    rec = TaskRecord(task_id=task_id, state="processing", kind="refund", message=message)
    _TASKS[task_id] = rec
    return rec

```

» main.py

```

"""Week 9 - Hosting, Deployment & Scale.
Customer Service Swarm: fast-path FAQ answers + durable long-running refund tasks
with status polling. Run: uvicorn app.main:app --reload
"""
from fastapi import BackgroundTasks, FastAPI, HTTPException
from app.service import (
    SupportRequest,
    SupportResponse,
    TaskRecord,
    answer_faq,
    get_settings,
    get_task,
    is_refund,
    process_refund,
    start_refund_task,
)
settings = get_settings()
app = FastAPI(title="Week 9 - Hosting & Scale (CS Swarm)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {"status": "ok", "week": "9", "backend": "mock"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week09-hosting-scale",
        "endpoint": "/api/v1/support",
        "docs": "/docs",
    }
@app.get("/api/v1/scale", tags=["week09"])
def scale_config() -> dict[str, int]:
    """Expose the concurrency knobs that drive KEDA autoscaling decisions."""
    return {
        "max_concurrent_sessions": settings.max_concurrent_sessions,
        "worker_count": settings.worker_count,
    }
@app.post("/api/v1/support", response_model=SupportResponse, tags=["week09"])
def support(payload: SupportRequest, background: BackgroundTasks) -> SupportResponse:
    """Fast path answers inline; refunds spawn a durable background task."""
    if is_refund(payload.customer_message):

```

```

        rec = start_refund_task(payload.customer_message, payload.order_id)
        background.add_task(process_refund, rec.task_id, payload.customer_message, payload.order_id)
        return SupportResponse(kind="task", task_id=rec.task_id, state=rec.state)
    return SupportResponse(kind="answer", answer=answer_faq(payload.customer_message))
@app.get("/api/v1/tasks/{task_id}", response_model=TaskRecord, tags=["week09"])
def task_status(task_id: str) -> TaskRecord:
    rec = get_task(task_id)
    if rec is None:
        raise HTTPException(status_code=404, detail="Task not found.")
    return rec

```

Week 10 - Observability, Evaluation & Governance

Week 10 - Observability, Evaluation & Governance

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no az login, keys, or .env required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Trace, **evaluate**, and govern agents in production.

Enterprise use case - "Regulated Advice Quality Monitoring" (Financial Services)

Every interaction is traced end-to-end, scored for groundedness/relevance/safety, and **flagged** when quality drops - feeding a dashboard auditors can review.

[x] What this repo implements

- **Tracing** - a trace id + span per request (OpenTelemetry -> Application Insights when configured; in-memory trace log otherwise).
- **Evaluation** - deterministic scoring (groundedness, relevance, safety, coherence). Foundry Evaluations in prod.
- **Governance** - answers below `QUALITY_THRESHOLD` are flagged; aggregate metrics at `/api/v1/metrics`.

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/advise \
-H "Content-Type: application/json" \
-d '{"client_question": "Should I move my pension this year?"}'
curl http://127.0.0.1:8000/api/v1/metrics
curl http://127.0.0.1:8000/api/v1/traces
```

Run tests: `pytest -q`

Endpoints

Method	Path	Purpose
POST	<code>/api/v1/advise</code>	Answer + per-request eval scores
GET	<code>/api/v1/metrics</code>	Aggregate quality + flagged rate
GET	<code>/api/v1/traces</code>	Recent traces

Enable App Insights

Set `APPLICATIONINSIGHTS_CONNECTION_STRING` to export spans via OpenTelemetry. GET `/health` reports

`"tracing": "on"`.

Architect's lens

- Distributed tracing across agent hops and tool calls (trace/span per agent).
- Offline eval (CI regression gates) vs. online eval (production sampling).
- Golden datasets, A/B prompts, drift detection; cost & latency per token / per tool.

Tech stack

OpenTelemetry, Azure Application Insights, Azure Monitor, Foundry Evaluation SDK, Content Safety, FastAPI.

Series

Prev: [Week 9](#) ·

Next: [Week 11 - Security](#) ·

[All labs](#)

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: [agentic-ai-azure-week10-observability.pptx](#)

Prefer PDF? Download the **handout (slides + speaker notes)**:

[agentic-ai-azure-week10-observability-handout.pdf](#)

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» [service.py](#)

```
"""Week 10 - Observability, Evaluation & Governance: Advice Quality Monitoring.
Demonstrates the three pillars of production agent ops:
* **Tracing** - each request gets a trace id and a recorded span (OpenTelemetry -&gt;
  Application Insights when configured; an in-memory trace log otherwise).
* **Evaluation** - every answer is scored for groundedness / relevance / safety /
  coherence (Foundry Evaluations in prod; deterministic heuristics here).
* **Governance** - answers that fall below thresholds are flagged, and aggregate
  quality metrics are exposed for a dashboard/alerts.
"""
from __future__ import annotations
import re
import time
import uuid
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    applicationinsights_connection_string: str = ""
    quality_threshold: float = 0.6
    @property
    def tracing_enabled(self) -> bool:
        return bool(self.applicationinsights_connection_string)
    @lru_cache
    def get_settings() -> Settings:
        return Settings()
# schemas
class AdviseRequest(BaseModel):
    client_question: str = Field(..., min_length=1, description="A client question for the advice agent.")
class Scores(BaseModel):
    groundedness: float
    relevance: float
    safety: float
    coherence: float
class AdviseResponse(BaseModel):
```

```

    answer: str
    scores: Scores
    flagged: bool
    trace_id: str
    mode: str
# grounding context + safety rules
_CONTEXT = {
    "pension": "Pension transfers depend on your risk tolerance, fees, and time horizon.",
    "fund": "Higher-growth funds carry more volatility and are better suited to longer horizons.",
    "savings": "Emergency savings should typically cover three to six months of expenses.",
}
_BANNED = ("guaranteed returns", "get rich", "risk-free profit")
_WORD_RE = re.compile(r"[a-z]{3,}")
def _tokens(text: str) -> set[str]:
    return set(_WORD_RE.findall(text.lower()))
# trace + metrics store (Application Insights in prod)
_TRACES: list[dict] = []
def _record_trace(trace_id: str, question: str, scores: Scores, flagged: bool, latency_ms: float) -> None:
    _TRACES.append(
        {
            "trace_id": trace_id,
            "question": question,
            "scores": scores.model_dump(),
            "flagged": flagged,
            "latency_ms": round(latency_ms, 2),
        }
    )
    _emit_span(trace_id, scores, flagged) # best-effort OTel export
def _emit_span(trace_id: str, scores: Scores, flagged: bool) -> None:
    """Best-effort OpenTelemetry span; no-ops if OTel/App Insights not set up."""
    if not get_settings().tracing_enabled:
        return
    try: # pragma: no cover - exercised only when OTel is installed/configured
        from opentelemetry import trace
        tracer = trace.get_tracer("advice-agent")
        with tracer.start_as_current_span("advise") as span:
            span.set_attribute("trace_id", trace_id)
            span.set_attribute("eval.groundedness", scores.groundedness)
            span.set_attribute("eval.flagged", flagged)
    except Exception:
        pass
def recent_traces(limit: int = 20) -> list[dict]:
    return _TRACES[-limit:]
def metrics_summary() -> dict:
    if not _TRACES:
        return {"count": 0, "flagged": 0, "flagged_rate": 0.0, "avg_scores": {}}
    n = len(_TRACES)
    flagged = sum(1 for t in _TRACES if t["flagged"])
    keys = ["groundedness", "relevance", "safety", "coherence"]
    avg = {k: round(sum(t["scores"][k] for t in _TRACES) / n, 3) for k in keys}
    return {"count": n, "flagged": flagged, "flagged_rate": round(flagged / n, 3), "avg_scores": avg}
}
# evaluation
def evaluate(question: str, answer: str, context: str | None) -> Scores:
    q, a = _tokens(question), _tokens(answer)
    relevance = round(len(q & a) / (len(q) or 1), 3)
    groundedness = 0.92 if context and context.lower() in answer.lower() else 0.45
    safety = 0.2 if any(b in answer.lower() for b in _BANNED) else 1.0
    coherence = 0.88 if 5 <= len(answer.split()) <= 80 else 0.55
    return Scores(groundedness=groundedness, relevance=relevance, safety=safety, coherence=coherence)
}
# backend
class AdviceBackend:
    mode = "mock"
    def advise(self, req: AdviseRequest) -> AdviseResponse:
        start = time.perf_counter()
        trace_id = uuid.uuid4().hex
        context = next((v for k, v in _CONTEXT.items() if k in req.client_question.lower()), None)
        if context:
            answer = f"{context} Consider speaking with an adviser for your specific situation."
        else:
            answer = "I don't have grounded guidance for that; please consult an adviser."
        scores = evaluate(req.client_question, answer, context)
        threshold = get_settings().quality_threshold
        flagged = any(v < threshold for v in scores.model_dump().values())
        latency = (time.perf_counter() - start) * 1000
        _record_trace(trace_id, req.client_question, scores, flagged, latency)
        return AdviseResponse(answer=answer, scores=scores, flagged=flagged, trace_id=trace_id, mode=self.mode)

```

```
def get_backend() -> AdviceBackend:
    return AdviceBackend()
```

» main.py

```
"""Week 10 - Observability, Evaluation & Governance.
Advice Quality Monitoring: every answer is traced, evaluated, and flagged when it
drops below quality thresholds; aggregate metrics feed a dashboard. Run: uvicorn app.main:app --rel
oad
"""
from fastapi import FastAPI
from app.service import (
    AdviseRequest,
    AdviseResponse,
    get_backend,
    get_settings,
    metrics_summary,
    recent_traces,
)
settings = get_settings()
app = FastAPI(title="Week 10 - Observability (Advice Quality)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {"status": "ok", "week": "10", "tracing": "on" if settings.tracing_enabled else "off"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week10-observability",
        "endpoint": "/api/v1/advise",
        "docs": "/docs",
    }
@app.post("/api/v1/advise", response_model=AdviseResponse, tags=["week10"])
def advise(payload: AdviseRequest) -> AdviseResponse:
    return get_backend().advise(payload)
@app.get("/api/v1/metrics", tags=["week10"])
def metrics() -> dict:
    """Aggregate quality metrics for the dashboard / alerting."""
    return metrics_summary()
@app.get("/api/v1/traces", tags=["week10"])
def traces() -> dict:
    """Recent per-request traces (groundedness/relevance/safety/coherence)."""
    return {"traces": recent_traces()}
```

Week 11 - Enterprise Security & Compliance

Week 11 - Enterprise Security & Compliance

> **Run in VS Code - no Azure needed.** pip install -r requirements.txt, then uvicorn app.main:app --reload and open <http://127.0.0.1:8000/docs>. Runs in **mock mode** by default - no az login, keys, or .env required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](https://github.com/azure-agentic-ai-masterclass).

Learning goal

Secure agents with identity, RBAC, **least privilege**, and audit.

Enterprise use case - "HR Self-Service Agent with Least Privilege" (Enterprise / Public Sector)

Employees ask about benefits/payroll. The agent acts **on behalf of** the signed-in user, so it only sees data that user is entitled to. All accesses are audited.

[x] What this repo implements

- **Identity** - a FastAPI dependency resolves the caller. Dev mode uses X-Debug-User/X-Debug-Roles

headers; prod validates a **Microsoft Entra ID** bearer JWT (signature/audience/issuer, lazy-imported).

- **Least privilege (OBO)** - you can read your own record; reading another employee's requires a manager role, else **403**.

- **Audit** - every access (allowed or denied) is logged at GET /api/v1/audit.

Quick start (dev identity mode)

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
# Self-service (allowed)
curl -X POST http://127.0.0.1:8000/api/v1/hr/ask \
  -H "Content-Type: application/json" -H "X-Debug-User: alice" \
  -d '{"question": "How many vacation days do I have?"}'
# Another employee without manager role -> 403
curl -X POST http://127.0.0.1:8000/api/v1/hr/ask \
  -H "Content-Type: application/json" -H "X-Debug-User: alice" \
  -d '{"question": "vacation?", "target_user": "bob"}'
# Manager (allowed)
curl -X POST http://127.0.0.1:8000/api/v1/hr/ask \
  -H "Content-Type: application/json" -H "X-Debug-User: carol" -H "X-Debug-Roles: manager" \
  -d '{"question": "vacation?", "target_user": "bob"}'
```

Run tests: pytest -q

Wire the real Entra ID backend

The Entra JWT validation is **real and verified** - tests/test_jwt.py exercises the full signature/audience/issuer/expiry path with a self-signed key (only the JWKS network lookup is stubbed). To enforce it against a live tenant:

1. Register the API app (Azure CLI)

[bash]

```
az login
# Create an app registration for THIS API
az ad app create --display-name "hr-agent-api" --sign-in-audience AzureADMyOrg
```

```

APP_ID=$(az ad app list --display-name "hr-agent-api" --query "[0].appId" -o tsv)
# Expose an API + an app role used for least privilege
az ad app update --id $APP_ID --identifier-uri "api://$APP_ID"
# In the portal (App registrations -> App roles) add an app role "manager" (value: manager),
# or define it via the app manifest. Assign it to users/groups under Enterprise applications.

```

Note your **tenant id** (`az account show --query tenantId -o tsv`) and `$APP_ID`.

2. Configure .env

```

AZURE_TENANT_ID=<tenant-guid>;
AZURE_CLIENT_ID=<APP_ID>;
# Optional overrides:
# AZURE_API_AUDIENCE=api://<APP_ID>; # default
# AZURE_ISSUER=https://login.microsoftonline.com/<tenant>/v2.0 # default (v2 tokens)

```

Common gotcha: if your access tokens are **v1.0** (default unless the manifest sets

`accessTokenAcceptedVersion: 2`), set

`AZURE_ISSUER=https://sts.windows.net/<tenant>/` and the audience

to the GUID. The validator reports the exact mismatch in the 401 detail.

3. Run - now Entra-enforced

[bash]

```

uvicorn app.main:app --reload # GET /health -> "auth": "entra"

```

The X-Debug- * headers are now ignored; requests must carry `Authorization: Bearer <token>`.

App roles in

the token's roles claim drive least-privilege (a manager role can read reports).

4. Get a token to test

[bash]

```

# Client-credentials example (app-only); for user OBO use the on-behalf-of flow.
az account get-access-token --resource "api://<APP_ID>" --query accessToken -o tsv

```

Then: `curl -H "Authorization: Bearer <token>" ...` Next step in the lab is **OAuth2**

On-Behalf-Of so

downstream tools run with the user's identity, with all secrets in **Key Vault** + Managed Identity.

Architect's lens

- Identity boundaries: app vs. user vs. tool identity.
- Data-exfiltration & prompt-injection defenses (tool allow-lists, output filtering).
- RBAC + Conditional Access; private networking (Private Endpoints, VNet).
- Audit/compliance: who did what, retention, Microsoft Purview lineage.

Structure

```

app/service.py # settings, identity dependency, Entra JWT validation, least-privilege + audit
app/main.py # /api/v1/hr/ask (Depends(get_identity)) + /api/v1/audit
tests/test_app.py # dev-identity mode (headers)
tests/test_jwt.py # real Entra JWT validation (self-signed key, JWKS stubbed)

```

Tech stack

Microsoft Entra ID, OAuth2/OBO, MSAL, PyJWT, Key Vault, Managed Identity, Content Safety, Purview.

Series

Prev: Week 10 ·

Next: Week 12 - Capstone ·

All labs

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-week11-security.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-week11-security-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `service.py`

```
"""Week 11 - Enterprise Security & Compliance: HR Self-Service Agent.
Demonstrates least-privilege agent access:
* **Identity** - requests carry an identity. In dev/mock mode it comes from
  ``X-Debug-User`` / ``X-Debug-Roles`` headers; in prod a **Microsoft Entra ID**
  bearer JWT is validated (lazy-imported).
* **On-Behalf-Of / least privilege** - the agent only returns data the signed-in
  user is entitled to; reading another employee's record requires a manager role.
* **Audit** - every access (allowed or denied) is logged.
"""
from __future__ import annotations
from functools import lru_cache
from fastapi import HTTPException, Request
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    azure_tenant_id: str = ""
    azure_client_id: str = "" # the API's app (client) id
    azure_api_audience: str = "" # override; defaults to api://<client_id>;
    azure_issuer: str = "" # override; defaults to the v2.0 issuer for the tenant
    @property
    def require_auth(self) -> bool:
        """Real Entra JWT validation is enforced when tenant + client are set."""
        return bool(self.azure_tenant_id and self.azure_client_id)
    @property
    def entra_audience(self) -> str:
        return self.azure_api_audience or f"api://{self.azure_client_id}"
    @property
    def entra_issuer(self) -> str:
        return self.azure_issuer or f"https://login.microsoftonline.com/{self.azure_tenant_id}/v2.0"
    @property
    def jwks_uri(self) -> str:
        return f"https://login.microsoftonline.com/{self.azure_tenant_id}/discovery/v2.0/keys"
@lru_cache
def get_settings() -> Settings:
    return Settings()
# identity
class Identity(BaseModel):
    user: str
    roles: list[str] = Field(default_factory=list)
def get_identity(request: Request) -> Identity:
    """FastAPI dependency that resolves the caller identity (Entra or dev headers)."""
    settings = get_settings()
    auth = request.headers.get("authorization")
    if settings.require_auth:
        if not auth or not auth.lower().startswith("bearer "):
            raise HTTPException(status_code=401, detail="Missing bearer token.")
        claims = _validate_entra_jwt(auth.split(" ", 1)[1])
        roles = claims.get("roles", []) or []
        return Identity(user=claims.get("preferred_username") or claims.get("sub", "unknown"), roles
=roles)
    # Dev/mock mode: identity from headers (never use in production).
    user = request.headers.get("x-debug-user")
    if not user:
        raise HTTPException(status_code=401, detail="No identity. Set X-Debug-User (dev) or configur
e Entra.")
    roles = [r.strip() for r in request.headers.get("x-debug-roles", "").split(",") if r.strip()]
    return Identity(user=user, roles=roles)
@lru_cache
def _jwks_client(): # pragma: no cover - network; cached across requests
    from jwt import PyJWKClient
    return PyJWKClient(get_settings().jwks_uri)
def _get_signing_key(token: str):
```

```

        """Resolve the RSA public key for a token from Entra's JWKS (by `kid`).
        Isolated so tests can inject a key without hitting the network.
        """
        return _jwks_client().get_signing_key_from_jwt(token).key
def _validate_entra_jwt(token: str) -> dict:
    """Validate a Microsoft Entra ID access token: signature, audience, issuer, expiry."""
    import jwt
    s = get_settings()
    try:
        return jwt.decode(
            token,
            _get_signing_key(token),
            algorithms=["RS256"],
            audience=s.entra_audience,
            issuer=s.entra_issuer,
        )
    except jwt.PyJWTError as exc:
        raise HTTPException(status_code=401, detail=f"Invalid token: {exc}") from exc
# HR data + audit (Key Vault / downstream API + Purview in prod)
_HR = {
    "alice": {"vacation_days": 12, "manager": "carol", "salary_band": "B"},
    "bob": {"vacation_days": 5, "manager": "carol", "salary_band": "C"},
    "carol": {"vacation_days": 20, "manager": None, "salary_band": "A"},
}
_AUDIT: list[dict] = []
def audit_log() -> list[dict]:
    return _AUDIT[-50:]
# schemas
class HRRequest(BaseModel):
    question: str = Field(..., min_length=1, description="The employee's HR question.")
    target_user: str | None = Field(default=None, description="Whose record (defaults to self).")
class HRResponse(BaseModel):
    answer: str
    acting_as: str
    on_behalf_of: str
    entitled: bool
    mode: str
# backend
def answer_hr(req: HRRequest, identity: Identity) -> HRResponse:
    target = (req.target_user or identity.user).lower()
    is_self = target == identity.user.lower()
    is_manager = "manager" in identity.roles
    entitled = is_self or is_manager
    _AUDIT.append(
        {"actor": identity.user, "target": target, "action": "hr.read", "allowed": entitled}
    )
    if not entitled:
        raise HTTPException(status_code=403, detail="Least privilege: you may only access your own r
ecord.")
    record = _HR.get(target)
    if not record:
        answer = f"No HR record found for '{target}'."
    elif "vacation" in req.question.lower() or "leave" in req.question.lower():
        answer = f"{target} has {record['vacation_days']} vacation days remaining."
    elif "salary" in req.question.lower() or "pay" in req.question.lower():
        answer = f"{target}'s salary band is {record['salary_band']}."
    else:
        answer = f"I can help with leave and pay-band questions for {target}."
    return HRResponse(
        answer=answer,
        acting_as=identity.user,
        on_behalf_of=target,
        entitled=True,
        mode="entra" if get_settings().require_auth else "dev",
    )

```

» main.py

```

"""Week 11 - Enterprise Security & Compliance.
HR Self-Service Agent with least-privilege access, Entra ID identity (dev headers
in mock mode), and an audit log. Run: uvicorn app.main:app --reload
"""
from fastapi import Depends, FastAPI
from app.service import (
    HRRequest,
    HRResponse,
    Identity,
    answer_hr,
    audit_log,

```

```

        get_identity,
        get_settings,
    )
    settings = get_settings()
    app = FastAPI(title="Week 11 - Security (HR Least-Privilege Agent)", version="0.2.0")
    @app.get("/health", tags=["health"])
    def health() -> dict[str, str]:
        return {"status": "ok", "week": "11", "auth": "entra" if settings.require_auth else "dev"}
    @app.get("/", tags=["root"])
    def root() -> dict[str, str]:
        return {
            "service": "agentic-ai-azure-week11-security",
            "endpoint": "/api/v1/hr/ask",
            "auth": "entra" if settings.require_auth else "dev",
            "docs": "/docs",
        }
    @app.post("/api/v1/hr/ask", response_model=HRResponse, tags=["week11"])
    def hr_ask(payload: HRRequest, identity: Identity = Depends(get_identity)) -> HRResponse:
        """Answer HR questions, enforcing least-privilege on whose record is read."""
        return answer_hr(payload, identity)
    @app.get("/api/v1/audit", tags=["week11"])
    def audit() -> dict:
        """Audit trail of accesses (allowed and denied)."""
        return {"audit": audit_log()}

```

Week 12 - Capstone: Build, Demo & Review

Week 12 - Capstone: Build, Demo & Review

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. Runs in **mock mode** by default - no `az login`, keys, or `.env` required. Wiring real Azure (below) is optional.

Standalone lab from the **Agentic AI on Azure - Enterprise Master Class**.

Course hub: [azure-agentic-ai-masterclass](#).

Learning goal

Ship a production-shaped, **multi-agent** application end-to-end.

Capstone scenario - "Customer Onboarding Automation" (Banking/Telecom)

A staged pipeline of cooperating agents: **Intake -> KYC/Compliance -> Risk -> Provisioning**, each with its own status, producing a single onboarding decision. (The other course scenarios - transaction compliance, lead prioritization, document generation - follow the same shape.)

[x] What this repo implements

A sequential multi-agent pipeline that reuses patterns from across the course:

Stage	Borrows from	Behavior
intake	Wk 1-2	extract name, ID, amount
kyc	Wk 6-7	screening guardrail (sanction/watchlist/PEP)
risk	Wk 6-7	flag high-value cases for review
provisioning	Wk 5 tools	create the account when cleared

Decision: onboarded / needs_review / rejected. Runnable offline; `FOUNDRY_PROJECT_ENDPOINT` toggles the reported mode.

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/capstone \
-H "Content-Type: application/json" \
-d '{"applicant_text": "Name: John Doe, ID-AB7788, requesting a $20,000 account."}'
```

Run tests: `pytest -q`

Capstone deliverables (extend this)

- 2-4 coordinating agents with tools via MCP/OpenAPI.
- Grounding on enterprise data with permission-aware retrieval (Wk 8).
- Containerized FastAPI deployed to Container Apps/AKS with CI/CD (Wk 9).
- Identity, observability, evaluation, and guardrails wired in (Wk 10-11).
- Architecture write-up + live demo.

Architect's lens

Defend your design - pattern choice, deployment target, cost model, security posture, and day-2 operations.

Tech stack

Everything: multi-agent orchestration + MCP/OpenAPI, RAG grounding, containerized FastAPI + CI/CD, identity, observability, evaluation.

Series

Prev: [Week 11](#) ·

[Course hub: azure-agentic-ai-masterclass](#) ·

[All labs](#)

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: [agentic-ai-azure-week12-capstone.pptx](#)

Prefer PDF? Download the **handout (slides + speaker notes)**:

[agentic-ai-azure-week12-capstone-handout.pdf](#)

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» [service.py](#)

```
"""Week 12 - Capstone: Customer Onboarding Automation (multi-agent).
Brings the course together in one pipeline that mirrors patterns from earlier weeks:
* **Intake agent** (Wk1/2) - extract structured fields.
* **KYC/Compliance agent** (Wk6-7) - screening with a guardrail.
* **Risk agent** (Wk6-7) - score by requested amount.
* **Provisioning agent** (Wk5 tools) - create the account when cleared.
Runs as a sequential staged pipeline with per-stage status. Deterministic + offline;
`FOUNDRY_PROJECT_ENDPOINT` toggles the reported mode. Other capstone scenarios
(transaction compliance, lead prioritization, document generation) follow the same shape.
"""
from __future__ import annotations
import re
import uuid
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    foundry_project_endpoint: str = ""
    foundry_model_name: str = "gpt-4o"
    review_amount_threshold: float = 100_000.0
    @property
    def use_foundry(self) -> bool:
        return bool(self.foundry_project_endpoint)
    @lru_cache
    def get_settings() -> Settings:
        return Settings()
# schemas
class CapstoneRequest(BaseModel):
    scenario: str = Field(default="customer_onboarding", description="Capstone scenario id.")
    applicant_text: str = Field(..., min_length=1, description="Applicant / case details.")
class Stage(BaseModel):
    name: str
    status: str # ok | flagged | skipped
    detail: str
class CapstoneResponse(BaseModel):
    scenario: str
    case_id: str
    decision: str # onboarded | needs_review | rejected
    stages: list[Stage]
```

```

        extracted: dict
        mode: str
# agents (stages)
def intake_stage(text: str) -> tuple[dict, Stage]:
    name = re.search(r"name[:\s]+([A-Za-z.\- ]+)?(?:,|$)", text, re.IGNORECASE)
    idn = re.search(r"\bID[:\s]*([A-Z0-9]{4,})\b", text, re.IGNORECASE)
    amount_m = re.search(r"\$\s*([\d,]+)\s*(k)?", text)
    amount = None
    if amount_m:
        amount = float(amount_m.group(1).replace(",", "")) * (1000 if (amount_m.group(2) or "").lower
r() == "k" else 1)
    extracted = {
        "name": name.group(1).strip() if name else None,
        "id_number": idn.group(1).upper() if idn else None,
        "amount": amount,
    }
    return extracted, Stage(name="intake", status="ok", detail=f"Extracted {extracted}.")
def kyc_stage(text: str, extracted: dict) -> Stage:
    flagged = any(w in text.lower() for w in ("sanction", "watchlist", "pep"))
    if not extracted.get("id_number"):
        return Stage(name="kyc", status="flagged", detail="Missing ID number.")
    return Stage(
        name="kyc",
        status="flagged" if flagged else "ok",
        detail="Screening hit." if flagged else "KYC/AML clear.",
    )
def risk_stage(extracted: dict, threshold: float) -> Stage:
    amount = extracted.get("amount") or 0
    if amount > threshold:
        return Stage(name="risk", status="flagged", detail=f"High value (${amount:,.0f}) needs review.")
    return Stage(name="risk", status="ok", detail="Within auto-approve limit.")
def provisioning_stage(decision: str) -> Stage:
    if decision == "onboarded":
        return Stage(name="provisioning", status="ok", detail="Account provisioned.")
    return Stage(name="provisioning", status="skipped", detail=f"Not provisioned (decision={decision
}).")
def run_pipeline(req: CapstoneRequest) -> CapstoneResponse:
    settings = get_settings()
    extracted, s_intake = intake_stage(req.applicant_text)
    s_kyc = kyc_stage(req.applicant_text, extracted)
    s_risk = risk_stage(extracted, settings.review_amount_threshold)
    if s_kyc.status == "flagged" and "hit" in s_kyc.detail.lower():
        decision = "rejected"
    elif s_kyc.status == "flagged" or s_risk.status == "flagged":
        decision = "needs_review"
    else:
        decision = "onboarded"
    s_prov = provisioning_stage(decision)
    return CapstoneResponse(
        scenario=req.scenario,
        case_id=f"CASE-{uuid.uuid4().hex[:8].upper()}",
        decision=decision,
        stages=[s_intake, s_kyc, s_risk, s_prov],
        extracted=extracted,
        mode="foundry" if settings.use_foundry else "mock",
    )

```

» main.py

```

"""Week 12 - Capstone: Build, Demo & Review.
Customer Onboarding Automation: a multi-agent staged pipeline (intake -> KYC ->
risk -> provisioning). Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.service import CapstoneRequest, CapstoneResponse, get_settings, run_pipeline
settings = get_settings()
app = FastAPI(title="Week 12 - Capstone (Customer Onboarding)", version="0.2.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {"status": "ok", "week": "12", "backend": "foundry" if settings.use_foundry else "mock"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-week12-capstone",
        "endpoint": "/api/v1/capstone",
        "backend": "foundry" if settings.use_foundry else "mock",
        "docs": "/docs",
    }

```

```
@app.post("/api/v1/capstone", response_model=CapstoneResponse, tags=["week12"])
def capstone(payload: CapstoneRequest) -> CapstoneResponse:
    return run_pipeline(payload)
```

Part III - Framework Comparison Projects

LangGraph on Azure - Support Triage Graph

LangGraph on Azure - Support Triage Graph

*Companion project to the *Agentic AI on Azure - Enterprise Master Class*, showing the **LangGraph** framework alongside the Microsoft Agent Framework labs.*

Course hub: [azure-agentic-ai-masterclass](#).

*> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. The LangGraph graph runs **for real** offline; Azure OpenAI is optional.*

What it shows

How to build an agent as a **graph** with LangGraph:

typed shared **state**, **nodes**, and **conditional edges** - the graph executes for real even without an LLM, and optionally calls **Azure OpenAI** to polish the answer.

The graph

```
START -> classify -> (high severity) -> escalate
              (normal)              -> retrieve -> respond -> END
```

- **classify** - assess severity + category
- **retrieve** - pull the KB article for the category (a "tool" node)
- **respond** - finalize the answer (optionally rephrased by Azure OpenAI)
- **escalate** - route urgent tickets to a human (guardrail branch)

The response includes the path of nodes actually visited, so you can see the routing.

Quick start

[bash]

```
python -m venv .venv && .\venv\Scripts\Activate.ps1 # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/triage \
-H "Content-Type: application/json" \
-d '{"message": "Production is down, urgent!"}' # -> escalate path
curl -X POST http://127.0.0.1:8000/api/v1/triage \
-H "Content-Type: application/json" \
-d '{"message": "I forgot my password"}' # -> retrieve -> respond
```

Run tests: `pytest -q`

Optional: Azure OpenAI

Set `AZURE_OPENAI_ENDPOINT` + `AZURE_OPENAI_API_KEY` (+ deployment) in `.env`. The respond node then

uses `langchain-openai`'s `AzureChatOpenAI` to rephrase the grounded answer. `GET /health` reports

`"mode": "azure"`. Without it, the graph still runs end-to-end.

Framework comparison

New to these frameworks? See **COMPARISON.md** - a side-by-side of

Microsoft Agent Framework / Foundry vs. LangGraph vs. Semantic Kernel (abstractions, control flow, state, tools, Azure fit, the same agent in each, and how to choose).

How it maps to the course

This is the **graph-based orchestration** counterpart to

Weeks 6-7 (multi-agent).

LangGraph and the Microsoft Agent Framework solve similar problems (stateful, branching agent flows) with different APIs - see COMPARISON.md.

Tech stack

LangGraph, LangChain Core, langchain-openai (Azure OpenAI), FastAPI, Pydantic v2.

Structure

```
app/graph.py      # StateGraph: state, nodes, conditional edges, compiled graph
app/main.py       # FastAPI app + POST /api/v1/triage
tests/test_app.py
```

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-langgraph.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-langgraph-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» graph.py

```
"""LangGraph - IT Support Triage Graph.
A real **LangGraph** ``StateGraph`` that demonstrates nodes, conditional edges, and
shared state. It runs **fully offline** (the nodes are plain Python), and optionally
uses **Azure OpenAI** (via ``langchain-openai``) to polish the final answer.
Graph:
    START -> classify -> (escalate | retrieve) -> respond -> END
"""
from __future__ import annotations
from functools import lru_cache
from typing import TypedDict
from langgraph.graph import END, START, StateGraph
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict

# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    azure_openai_endpoint: str = ""
    azure_openai_api_key: str = ""
    azure_openai_deployment: str = "gpt-4o"
    azure_openai_api_version: str = "2024-10-21"
    @property
    def use_azure(self) -> bool:
        return bool(self.azure_openai_endpoint and self.azure_openai_api_key)
    @lru_cache
    def get_settings() -> Settings:
        return Settings()

# schemas
class TriageRequest(BaseModel):
    message: str = Field(..., min_length=1, description="Incoming support ticket / user message.")
class TriageResponse(BaseModel):
    answer: str
    category: str
    severity: str
    escalated: bool
    path: list[str] = Field(..., description="Graph nodes visited, in order.")
    mode: str

# knowledge / rules
KB = {
    "vpn": "Reset your VPN client, then reconnect with your corporate credentials.",
    "password": "Use the self-service portal to reset your password; locked accounts auto-unlock in 30 min.",
```

```

    "email": "Restart Outlook and re-add the account; check service health if many users are affected.",
}
}
ESCALATE_SIGNALS = ("urgent", "down", "outage", "breach", "production", "asap")
# graph state + nodes
class GraphState(TypedDict):
    message: str
    category: str
    severity: str
    escalated: bool
    answer: str
    path: list[str]
def classify_node(state: GraphState) -> dict:
    msg = state["message"].lower()
    severity = "high" if any(s in msg for s in ESCALATE_SIGNALS) else "normal"
    category = next((k for k in KB if k in msg), "general")
    return {"severity": severity, "category": category, "path": [*state["path"], "classify"]}
def route_after_classify(state: GraphState) -> str:
    return "escalate" if state["severity"] == "high" else "retrieve"
def retrieve_node(state: GraphState) -> dict:
    return {"answer": KB.get(state["category"], ""), "path": [*state["path"], "retrieve"]}
def respond_node(state: GraphState) -> dict:
    base = state["answer"] or "I couldn't find a KB article; routing to the helpdesk queue."
    return {"answer": _maybe_azure(base, state["message"]), "escalated": False,
            "path": [*state["path"], "respond"]}
def escalate_node(state: GraphState) -> dict:
    return {"answer": "Escalating to a human engineer (time-sensitive).", "escalated": True,
            "path": [*state["path"], "escalate"]}
@lru_cache
def build_graph():
    """Compile the StateGraph once and reuse it."""
    g = StateGraph(GraphState)
    g.add_node("classify", classify_node)
    g.add_node("retrieve", retrieve_node)
    g.add_node("respond", respond_node)
    g.add_node("escalate", escalate_node)
    g.add_edge(START, "classify")
    g.add_conditional_edges("classify", route_after_classify,
                           {"escalate": "escalate", "retrieve": "retrieve"})
    g.add_edge("retrieve", "respond")
    g.add_edge("respond", END)
    g.add_edge("escalate", END)
    return g.compile()
def run_triage(message: str) -> GraphState:
    initial: GraphState = {
        "message": message, "category": "", "severity": "",
        "escalated": False, "answer": "", "path": [],
    }
    return build_graph().invoke(initial)
def _maybe_azure(text: str, message: str) -> str:
    """Optionally rephrase the answer with Azure OpenAI (lazy-imported)."""
    s = get_settings()
    if not s.use_azure:
        return text
    from langchain_openai import AzureChatOpenAI
    llm = AzureChatOpenAI(
        azure_endpoint=s.azure_openai_endpoint,
        api_key=s.azure_openai_api_key,
        azure_deployment=s.azure_openai_deployment,
        api_version=s.azure_openai_api_version,
        temperature=0,
    )
    resp = llm.invoke(
        f"You are an IT helpdesk agent. Rephrase this answer politely and concisely "
        f"for the user's message '{message}':\n\n{text}"
    )
    return resp.content

```

» main.py

```

"""LangGraph - IT Support Triage Graph (FastAPI service).
Runs a real LangGraph StateGraph. Works offline (mock mode); set Azure OpenAI
env vars to polish answers with a model. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.graph import TriageRequest, TriageResponse, get_settings, run_triage
settings = get_settings()
app = FastAPI(title="LangGraph - Support Triage Graph", version="0.1.0")
@app.get("/health", tags=["health"])

```

```

def health() -> dict[str, str]:
    return {"status": "ok", "framework": "langgraph", "mode": "azure" if settings.use_azure else "mock"}
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-langgraph",
        "endpoint": "/api/v1/triage",
        "mode": "azure" if settings.use_azure else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/triage", response_model=TriageResponse, tags=["langgraph"])
def triage(payload: TriageRequest) -> TriageResponse:
    result = run_triage(payload.message)
    return TriageResponse(
        answer=result["answer"],
        category=result["category"],
        severity=result["severity"],
        escalated=result["escalated"],
        path=result["path"],
        mode="azure" if settings.use_azure else "mock",
    )

```

Semantic Kernel on Azure - Claims Validation

Semantic Kernel on Azure - Claims Validation

*Companion project to the *Agentic AI on Azure - Enterprise Master Class*, showing the **Semantic Kernel** framework alongside the Microsoft Agent Framework labs.*

Course hub: [azure-agentic-ai-masterclass](#).

*> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. The SK kernel + plugin run **for real** offline; Azure OpenAI is optional.*

What it shows

How to build an agent capability with Semantic Kernel:

a **Kernel** with a native **plugin** (`PolicyPlugin`) exposing a `@kernel_function`. The plugin runs for real even without an LLM; when **Azure OpenAI** is configured, the kernel adds an `AzureChatCompletion` service and uses **automatic function calling** so the model invokes the plugin itself.

How it works

- `PolicyPlugin.lookup_policy` - a native `@kernel_function` that validates a policy number.
- **Mock mode (default)**: the service extracts the policy number and invokes the plugin via `kernel.invoke(plugin_name="policy", function_name="lookup_policy", ...)` - genuine SK execution.
- **Azure mode**: `kernel.invoke_prompt(..., FunctionChoiceBehavior.Auto())` lets the model decide to call the plugin; structured fields stay authoritative (re-derived from the plugin).

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1 # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/claims/validate \
-H "Content-Type: application/json" \
-d '{"claim_text": "Minor accident, policy POL-12345, front bumper."}'
```

Run tests: `pytest -q`. Policies: POL-12345/POL-67890 (active -> approved), POL-00001 (lapsed -> rejected), anything else (not found -> needs_review).

Optional: Azure OpenAI (automatic function calling)

Set `AZURE_OPENAI_ENDPOINT` + `AZURE_OPENAI_API_KEY` (+ deployment) in `.env`. The kernel registers an

`AzureChatCompletion` service and the model auto-calls the plugin. `GET /health` reports "mode": "azure".

Dependency note: Semantic Kernel needs a newer starlette than FastAPI 0.115 allows, so this repo uses

`fastapi>=0.136`. The Azure auto-function-calling import paths can vary across SK versions - verify against your installed `semantic-kernel` if you customize that path.

How it maps to the course

This is the Semantic Kernel counterpart to
Week 2 (Foundry claims intake)

and the Weeks 3-4 Agent Framework lab
- the same "tool/plugin the agent calls" idea, expressed in SK.

Tech stack

Semantic Kernel, Azure OpenAI (AzureChatCompletion), FastAPI, Pydantic v2.

Structure

```
app/kernel_app.py # Kernel, PolicyPlugin (kernel_function), validation logic
app/main.py       # FastAPI app + POST /api/v1/claims/validate
tests/test_app.py
```

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-semantic-kernel.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-semantic-kernel-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `kernel_app.py`

```
"""Semantic Kernel on Azure - Claims Validation with a native plugin.
A real **Semantic Kernel** ``Kernel`` with a native **plugin** (`PolicyPlugin`)
whose `@kernel_function` validates an insurance policy. The plugin runs **for real
offline** (no AI service needed). When Azure OpenAI is configured, the kernel adds
an `AzureChatCompletion` service and uses **automatic function calling** so the model
invokes the plugin itself.
"""
from __future__ import annotations
import json
import re
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
from semantic_kernel import Kernel
from semantic_kernel.functions import KernelArguments, kernel_function
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    azure_openai_endpoint: str = ""
    azure_openai_api_key: str = ""
    azure_openai_deployment: str = "gpt-4o"
    azure_openai_api_version: str = "2024-10-21"
    @property
    def use_azure(self) -> bool:
        return bool(self.azure_openai_endpoint and self.azure_openai_api_key)
    @lru_cache
    def get_settings() -> Settings:
        return Settings()
# schemas
class ClaimRequest(BaseModel):
    claim_text: str = Field(..., min_length=1, description="Free-text insurance claim.")
class ClaimValidation(BaseModel):
    policy_number: str | None
    valid: bool
    status: str
    coverage: list[str]
    decision: str # approved | needs_review | rejected
    mode: str
    invoked_via: str
    summary: str | None = None
# the Semantic Kernel native plugin
_POLICIES = {
    "POL-12345": {"holder": "Jordan Avery", "status": "active",
                  "coverage": ["collision", "liability", "roadside"], "deductible": 500.0},
```

```

        "POL-67890": {"holder": "Sam Rivera", "status": "active",
                     "coverage": ["property", "contents", "flood"], "deductible": 1000.0},
        "POL-00001": {"holder": "Lapsed Customer", "status": "lapsed", "coverage": [], "deductible": None},
    }
}
class PolicyPlugin:
    """A native SK plugin the kernel (or the model) can call."""
    @kernel_function(
        name="lookup_policy",
        description="Validate an insurance policy number; returns JSON with status, coverage, deductible.",
    )
    def lookup_policy(self, policy_number: str) -> str:
        rec = _POLICIES.get((policy_number or "").strip().upper())
        if rec is None:
            return json.dumps({"policy_number": policy_number, "valid": False, "status": "not_found",
                              "coverage": []})
        return json.dumps({
            "policy_number": policy_number,
            "valid": rec["status"] == "active",
            "status": rec["status"],
            "holder": rec["holder"],
            "coverage": rec["coverage"],
            "deductible": rec["deductible"],
        })
    @lru_cache
    def build_kernel() -> Kernel:
        """Build the kernel once: register the plugin (+ Azure chat service if configured)."""
        kernel = Kernel()
        kernel.add_plugin(PolicyPlugin(), plugin_name="policy")
        s = get_settings()
        if s.use_azure:
            from semantic_kernel.connectors.ai.open_ai import AzureChatCompletion
            kernel.add_service(
                AzureChatCompletion(
                    deployment_name=s.azure_openai_deployment,
                    endpoint=s.azure_openai_endpoint,
                    api_key=s.azure_openai_api_key,
                    api_version=s.azure_openai_api_version,
                )
            )
        return kernel
    _POLICY_RE = re.compile(r"\bPOL-\d{4,6}\b", re.IGNORECASE)
    def _extract_policy(text: str) -> str | None:
        m = _POLICY_RE.search(text)
        return m.group(0).upper() if m else None
    def _decide(data: dict) -> str:
        status = data.get("status")
        if status in (None, "not_found", "missing"):
            return "needs_review"
        return "approved" if data.get("valid") else "rejected"
    async def _plugin_validate(req: ClaimRequest) -> ClaimValidation:
        """Validate by invoking the SK native plugin via the kernel (real SK, no Azure)."""
        kernel = build_kernel()
        policy_number = _extract_policy(req.claim_text)
        result = await kernel.invoke(
            plugin_name="policy",
            function_name="lookup_policy",
            arguments=KernelArguments(policy_number=policy_number or ""),
        )
        data = json.loads(str(result))
        return ClaimValidation(
            policy_number=policy_number,
            valid=bool(data.get("valid")),
            status=data.get("status", "unknown"),
            coverage=data.get("coverage", []),
            decision=_decide(data),
            mode="mock",
            invoked_via="semantic-kernel native plugin",
        )
    async def _azure_validate(req: ClaimRequest) -> ClaimValidation:
        """Let the Azure chat model call the plugin via automatic function calling.
        Structured fields stay authoritative (re-derived from the plugin); the model
        contributes a natural-language summary. Lazy-imported; runs only in azure mode.
        """
        from semantic_kernel.connectors.ai.function_choice_behavior import FunctionChoiceBehavior
        from semantic_kernel.connectors.ai.open_ai import AzureChatPromptExecutionSettings
        kernel = build_kernel()
        settings = AzureChatPromptExecutionSettings(function_choice_behavior=FunctionChoiceBehavior.Auto

```

```

    ())
    summary = await kernel.invoke_prompt(
        f"Validate the policy in this claim and summarize the coverage:\n{req.claim_text}",
        arguments=KernelArguments(settings=settings),
    )
    base = await _plugin_validate(req)
    base.mode = "azure"
    base.invoked_via = "semantic-kernel + AzureChatCompletion (auto function calling)"
    base.summary = str(summary)
    return base
async def validate_claim(req: ClaimRequest) -> ClaimValidation:
    return await (_azure_validate(req) if get_settings().use_azure else _plugin_validate(req))

```

» main.py

```

"""Semantic Kernel on Azure - Claims Validation (FastAPI service).
Uses a real SK kernel + native plugin. Works offline (mock mode); set Azure OpenAI
env vars for automatic function calling. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.kernel_app import ClaimRequest, ClaimValidation, get_settings, validate_claim
settings = get_settings()
app = FastAPI(title="Semantic Kernel - Claims Validation", version="0.1.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "framework": "semantic-kernel",
        "mode": "azure" if settings.use_azure else "mock",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-semantic-kernel",
        "endpoint": "/api/v1/claims/validate",
        "mode": "azure" if settings.use_azure else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/claims/validate", response_model=ClaimValidation, tags=["semantic-kernel"])
async def validate(payload: ClaimRequest) -> ClaimValidation:
    return await validate_claim(payload)

```

Part IV - Semantic Kernel + Azure Data Services

SK + Azure PostgreSQL - Orders Assistant

Semantic Kernel + Azure PostgreSQL - Orders Assistant

*Companion project to the *Agentic AI on Azure - Enterprise Master Class*.*

Course hub: [azure-agentic-ai-masterclass](#).

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. The SK kernel + plugin run **for real** offline; Azure PostgreSQL is optional.

What it shows

A **Semantic Kernel** agent skill backed by a **relational database**. A native plugin (`OrdersPlugin.get_order_status`) reads from an `orders` table - an in-memory table offline, or **Azure Database for PostgreSQL** (psycopg) when configured.

How it works

- `OrdersPlugin.get_order_status(order_id)` - a `@kernel_function` invoked via the kernel.
- **Mock mode (default)**: reads from an in-memory `orders` dict.
- **Postgres mode**: `_fetch_order` runs `SELECT status, customer, total FROM orders WHERE order_id=%s`.

Quick start

[bash]

```
python -m venv .venv &&& .\.venv\Scripts\Activate.ps1 # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/orders/ask \
-H "Content-Type: application/json" -d '{"question": "where is order ORD-1001?"}'
```

Run tests: `pytest -q`. Sample orders: ORD-1001 (shipped), ORD-1002 (processing), ORD-2001 (delivered).

Wire Azure Database for PostgreSQL

[bash]

```
az postgres flexible-server create -g <rg> -n my-pg --tier Burstable --sku-name Standard_B1ms
# create an `orders` table: order_id (text pk), status (text), customer (text), total (numeric)
```

Set `POSTGRES_CONNECTION_STRING` in `.env` (use `sslmode=require`; prefer Entra/Managed Identity auth in production). GET `/health` reports `"store": "postgres"`.

Tech stack

Semantic Kernel, Azure Database for PostgreSQL (psycopg), FastAPI, Pydantic v2.

Structure

```
app/kernel_app.py # Kernel, OrdersPlugin, data access (mock | postgres)
app/main.py       # POST /api/v1/orders/ask
tests/test_app.py
```

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-sk-postgresql.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-sk-postgresql-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `kernel_app.py`

```
"""Semantic Kernel + Azure Database for PostgreSQL - Orders Assistant.
A real **Semantic Kernel** ``Kernel`` with a native plugin (`OrdersPlugin`) whose
`@kernel_function` looks up an order. The data accessor runs **offline** against an
in-memory table by default, and against **Azure Database for PostgreSQL** (via
`psycopg`, lazy-imported) when `POSTGRES_CONNECTION_STRING` is set.
"""

from __future__ import annotations
import json
import re
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
from semantic_kernel import Kernel
from semantic_kernel.functions import KernelArguments, kernel_function
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    postgres_connection_string: str = ""
    @property
    def use_postgres(self) -> bool:
        return bool(self.postgres_connection_string)
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class OrderQuery(BaseModel):
    question: str = Field(..., min_length=1, description="Natural-language order question.")
class OrderResult(BaseModel):
    order_id: str | None
    found: bool
    status: str
    customer: str | None = None
    total: float | None = None
    answer: str
    mode: str
    invoked_via: str
# data access (in-memory mock | Azure PostgreSQL)
_ORDERS = {
    "ORD-1001": {"status": "shipped", "customer": "Jordan Avery", "total": 129.50},
    "ORD-1002": {"status": "processing", "customer": "Sam Rivera", "total": 42.00},
    "ORD-2001": {"status": "delivered", "customer": "Lee Chen", "total": 318.75},
}
def _fetch_order(order_id: str) -> dict | None:
    s = get_settings()
    key = (order_id or "").strip().upper()
    if not s.use_postgres:
        return _ORDERS.get(key)
    # Real Azure Database for PostgreSQL lookup (lazy import).
    import psycopg
    with psycopg.connect(s.postgres_connection_string) as conn:
        row = conn.execute(
            "SELECT status, customer, total FROM orders WHERE order_id = %s", (key,)
        ).fetchone()
    if not row:
        return None
    return {"status": row[0], "customer": row[1], "total": float(row[2])}
# the Semantic Kernel native plugin
class OrdersPlugin:
    @kernel_function(name="get_order_status", description="Look up an order by id; returns JSON.")
    def get_order_status(self, order_id: str) -> str:
        rec = _fetch_order(order_id)
        if rec is None:
            return json.dumps({"order_id": order_id, "found": False, "status": "not_found"})
        return json.dumps({"order_id": order_id, "found": True, **rec})
@lru_cache
def build_kernel() -> Kernel:
```

```

        kernel = Kernel()
        kernel.add_plugin(OrdersPlugin(), plugin_name="orders")
        return kernel
_ORDER_RE = re.compile(r"\bORD-\d{3,6}\b", re.IGNORECASE)
def _extract_order(text: str) -> str | None:
    m = _ORDER_RE.search(text)
    return m.group(0).upper() if m else None
async def ask_orders(req: OrderQuery) -> OrderResult:
    kernel = build_kernel()
    order_id = _extract_order(req.question)
    result = await kernel.invoke(
        plugin_name="orders", function_name="get_order_status",
        arguments=KernelArguments(order_id=order_id or ""),
    )
    data = json.loads(str(result))
    if not order_id:
        answer = "Please include an order id like ORD-1001."
    elif not data.get("found"):
        answer = f"No order found for {order_id}."
    else:
        answer = f"Order {order_id} for {data['customer']} is {data['status']} (total ${data['total']:.2f})."
    return OrderResult(
        order_id=order_id,
        found=bool(data.get("found")),
        status=data.get("status", "unknown"),
        customer=data.get("customer"),
        total=data.get("total"),
        answer=answer,
        mode="postgres" if get_settings().use_postgres else "mock",
        invoked_via="semantic-kernel native plugin",
    )

```

» main.py

```

"""Semantic Kernel + Azure PostgreSQL - Orders Assistant (FastAPI service).
Real SK kernel + native plugin. Works offline (mock table); set
POSTGRES_CONNECTION_STRING for Azure Database for PostgreSQL.
Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.kernel_app import OrderQuery, OrderResult, ask_orders, get_settings
settings = get_settings()
app = FastAPI(title="Semantic Kernel + Azure PostgreSQL - Orders Assistant", version="0.1.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "framework": "semantic-kernel",
        "store": "postgres" if settings.use_postgres else "mock",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-sk-postgresql",
        "endpoint": "/api/v1/orders/ask",
        "store": "postgres" if settings.use_postgres else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/orders/ask", response_model=OrderResult, tags=["sk-postgresql"])
async def orders_ask(payload: OrderQuery) -> OrderResult:
    return await ask_orders(payload)

```

SK + Azure AI Search - Knowledge Assistant

Semantic Kernel + Azure AI Search - Knowledge Assistant (RAG)

*Companion project to the *Agentic AI on Azure - Enterprise Master Class*.*

Course hub: [azure-agentic-ai-masterclass](#).

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. The SK kernel + plugin run **for real** offline; Azure AI Search is optional.

What it shows

A **Semantic Kernel** retrieval skill for **RAG**. A native plugin

(`KnowledgePlugin.search_knowledge`) fetches grounding documents - from an in-memory corpus offline, or

Azure AI Search when configured. The assistant answers only from retrieved content and cites sources.

How it works

- `KnowledgePlugin.search_knowledge(query)` - a `@kernel_function` invoked via the kernel.
- **Mock mode (default)**: keyword scoring over an in-memory corpus.
- **Search mode**: `SearchClient.search(search_text=...)` against your index.
- Grounding guardrail: returns "I don't know" when nothing relevant is found.

Quick start

[bash]

```
python -m venv .venv && .\venv\Scripts\Activate.ps1 # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/ask \
-H "Content-Type: application/json" -d '{"question": "What is the return policy refund window?"}'
```

Run tests: `pytest -q`. Sample topics: returns, shipping, warranty.

Wire Azure AI Search

Create a search service + an index with `title` and `content` fields, then set

`AZURE_AI_SEARCH_ENDPOINT` + `AZURE_AI_SEARCH_KEY` (+ `AZURE_AI_SEARCH_INDEX`). GET `/health` reports

`"store": "search"`. (For vector/hybrid retrieval + ingestion, see the Week 8 lab.)

Tech stack

Semantic Kernel, Azure AI Search (`azure-search-documents`), FastAPI, Pydantic v2.

Structure

```
app/kernel_app.py # Kernel, KnowledgePlugin, retrieval (mock | AI Search)
app/main.py       # POST /api/v1/ask
tests/test_app.py
```

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-sk-ai-search.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

[agentic-ai-azure-sk-ai-search-handout.pdf](#)

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `kernel_app.py`

```
"""Semantic Kernel + Azure AI Search - Knowledge Assistant (RAG).
A real **Semantic Kernel** ``Kernel`` with a native plugin (`KnowledgePlugin`) that
retrieves grounding documents. It searches an in-memory corpus offline, or **Azure AI
Search** (`azure-search-documents`, lazy-imported) when configured. The assistant only
answers from retrieved content and cites sources.
"""
from __future__ import annotations
import json
import re
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
from semantic_kernel import Kernel
from semantic_kernel.functions import KernelArguments, kernel_function
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    azure_ai_search_endpoint: str = ""
    azure_ai_search_key: str = ""
    azure_ai_search_index: str = "product-knowledge"
    @property
    def use_search(self) -> bool:
        return bool(self.azure_ai_search_endpoint and self.azure_ai_search_key)
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class AskRequest(BaseModel):
    question: str = Field(..., min_length=1, description="A product/policy question.")
class Citation(BaseModel):
    title: str
    snippet: str
    score: float
class AskResponse(BaseModel):
    answer: str
    grounded: bool
    citations: list[Citation]
    mode: str
    invoked_via: str
# corpus + retrieval (in-memory mock | Azure AI Search)
_DOCS = [
    {"title": "Return Policy", "content": "Items can be returned within 30 days with a receipt "
                                         "for a full refund to the original payment method."},
    {"title": "Shipping", "content": "Standard shipping takes 3-5 business days; express shipping "
                                     "is next business day before 2pm."},
    {"title": "Warranty", "content": "Electronics include a 1-year limited warranty covering "
                                     "manufacturing defects, excluding accidental damage."},
]
_WORD_RE = re.compile(r"[a-z]{3,}")
_STOPWORDS = {
    "the", "and", "for", "with", "what", "how", "does", "are", "you", "your", "this",
    "that", "from", "can", "will", "was", "has", "have", "but", "not", "all", "any",
    "out", "our", "who", "why", "when", "which", "into", "its",
}
def _toks(text: str) -> set[str]:
    return {w for w in _WORD_RE.findall(text.lower()) if w not in _STOPWORDS}
def _search_docs(query: str, top: int = 3) -> list[dict]:
    s = get_settings()
    if not s.use_search:
        q = _toks(query)
        scored = []
        for d in _DOCS:
            overlap = q & _toks(d["title"] + " " + d["content"])
            score = round(len(overlap) / (len(q) or 1), 3)
            if score > 0:
                scored.append({"title": d["title"], "snippet": d["content"][:160], "score": score})
        scored.sort(key=lambda c: c["score"], reverse=True)
        return scored[:top]
```

```

# Real Azure AI Search (lazy import).
from azure.core.credentials import AzureKeyCredential
from azure.search.documents import SearchClient
sc = SearchClient(s.azure_ai_search_endpoint, s.azure_ai_search_index,
                  AzureKeyCredential(s.azure_ai_search_key))
rows = sc.search(search_text=query, top=top, select=["title", "content"])
return [{"title": r.get("title", "doc"), "snippet": (r.get("content", "") or "")[:160],
         "score": float(r.get("@search.score", 0.0))} for r in rows]
# the Semantic Kernel native plugin
class KnowledgePlugin:
    @kernel_function(name="search_knowledge", description="Search the knowledge base; returns JSON list.")
    def search_knowledge(self, query: str) -> str:
        return json.dumps(_search_docs(query))
@lru_cache
def build_kernel() -> Kernel:
    kernel = Kernel()
    kernel.add_plugin(KnowledgePlugin(), plugin_name="knowledge")
    return kernel
async def ask_knowledge(req: AskRequest) -> AskResponse:
    kernel = build_kernel()
    result = await kernel.invoke(
        plugin_name="knowledge", function_name="search_knowledge",
        arguments=KernelArguments(query=req.question),
    )
    hits = [Citation(**h) for h in json.loads(str(result))]
    mode = "search" if get_settings().use_search else "mock"
    if not hits:
        return AskResponse(
            answer="I don't know based on the available documents.",
            grounded=False, citations=[], mode=mode, invoked_via="semantic-kernel native plugin",
        )
    top = hits[0]
    return AskResponse(
        answer=f"{top.snippet} (Source: {top.title})",
        grounded=True, citations=hits, mode=mode, invoked_via="semantic-kernel native plugin",
    )

```

» main.py

```

"""Semantic Kernel + Azure AI Search - Knowledge Assistant (FastAPI service).
Real SK kernel + native retrieval plugin. Works offline (in-memory corpus); set
Azure AI Search env vars to query a real index. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.kernel_app import AskRequest, AskResponse, ask_knowledge, get_settings
settings = get_settings()
app = FastAPI(title="Semantic Kernel + Azure AI Search - Knowledge Assistant", version="0.1.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "framework": "semantic-kernel",
        "store": "search" if settings.use_search else "mock",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-sk-ai-search",
        "endpoint": "/api/v1/ask",
        "store": "search" if settings.use_search else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/ask", response_model=AskResponse, tags=["sk-ai-search"])
async def ask(payload: AskRequest) -> AskResponse:
    return await ask_knowledge(payload)

```

SK + Azure Cosmos DB - Conversation Memory

Semantic Kernel + Azure Cosmos DB - Durable Conversation Memory

*Companion project to the *Agentic AI on Azure - Enterprise Master Class*.*

Course hub: [azure-agentic-ai-masterclass](#).

> **Run in VS Code - no Azure needed.** `pip install -r requirements.txt`, then `uvicorn app.main:app --reload` and open `http://127.0.0.1:8000/docs`. The SK kernel + plugin run **for real** offline; Cosmos DB is optional.

What it shows

A **Semantic Kernel** memory skill backed by a **document database**. A native plugin (MemoryPlugin) remembers and recalls conversation turns - in-memory offline, or **Azure Cosmos DB** when configured, partitioned by `sessionId` so memory is durable and scales across instances.

How it works

- `MemoryPlugin.remember(session_id, message)` and `recall(session_id)` - @kernel_functions invoked

via the kernel.

- **Mock mode (default):** in-memory per-session list.
- **Cosmos mode:** each turn is an item; count via `SELECT VALUE COUNT(1)`, recent via `ORDER BY c.ts DESC`,

both scoped to the `sessionId` partition.

Quick start

[bash]

```
python -m venv .venv &&& .\venv\Scripts\Activate.ps1 # macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

[bash]

```
curl -X POST http://127.0.0.1:8000/api/v1/chat \
-H "Content-Type: application/json" -d '{"session_id": "demo", "message": "hello"}'
# call again with the same session_id -& memory_turns grows; recent lists the messages
```

Run tests: `pytest -q`

Wire Azure Cosmos DB

[bash]

```
az cosmosdb create -g <rg> -n my-sk-cosmos --default-consistency-level Session
az cosmosdb keys list -g <rg> -n my-sk-cosmos --type connection-strings \
--query "connectionStrings[0].connectionString" -o tsv
```

The app **auto-creates** the database (skmemory) and container (messages, partition key /sessionId). Set `COSMOS_CONNECTION_STRING` in `.env`; GET `/health` reports "memory": "cosmos".

Tech stack

Semantic Kernel, Azure Cosmos DB (azure-cosmos), FastAPI, Pydantic v2.

Structure

```
app/kernel_app.py # Kernel, MemoryPlugin, memory stores (in-memory | Cosmos)
app/main.py       # POST /api/v1/chat
tests/test_app.py
```

License

MIT - see LICENSE.

Teaching slides

Download the **7-slide deck** for classroom use: `agentic-ai-azure-sk-cosmosdb.pptx`

Prefer PDF? Download the **handout (slides + speaker notes)**:

`agentic-ai-azure-sk-cosmosdb-handout.pdf`

Slides: Title · Learning goal · Enterprise use case · Architecture/flow · Key concepts · Run it · Architect's takeaways.

Key source code

» `kernel_app.py`

```
"""Semantic Kernel + Azure Cosmos DB - Durable Conversation Memory.
A real **Semantic Kernel** ``Kernel`` with a native plugin (`MemoryPlugin`) that
remembers and recalls conversation turns. The store is in-memory offline, or **Azure
Cosmos DB** (`azure-cosmos`, lazy-imported) when `COSMOS_CONNECTION_STRING` is set -
so chat memory survives restarts and scales across instances (partitioned by sessionId).
"""
from __future__ import annotations
import json
from functools import lru_cache
from pydantic import BaseModel, Field
from pydantic_settings import BaseSettings, SettingsConfigDict
from semantic_kernel import Kernel
from semantic_kernel.functions import KernelArguments, kernel_function
# settings
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")
    app_env: str = "local"
    cosmos_connection_string: str = ""
    cosmos_database: str = "skmemory"
    cosmos_container: str = "messages"
    @property
    def use_cosmos(self) -> bool:
        return bool(self.cosmos_connection_string)
@lru_cache
def get_settings() -> Settings:
    return Settings()
# schemas
class ChatRequest(BaseModel):
    session_id: str = Field(default="default", description="Conversation/session id.")
    message: str = Field(..., min_length=1, description="The user's message.")
class ChatResponse(BaseModel):
    reply: str
    session_id: str
    memory_turns: int
    recent: list[str]
    mode: str
    invoked_via: str
# memory stores (in-memory mock | Azure Cosmos DB)
class InMemoryStore:
    def __init__(self) -> None:
        self._data: dict[str, list[str]] = {}
    def append(self, session_id: str, message: str) -> int:
        turns = self._data.setdefault(session_id, [])
        turns.append(message)
        return len(turns)
    def recent(self, session_id: str, n: int = 5) -> list[str]:
        return self._data.get(session_id, [])[-n:]
class CosmosStore:
    def __init__(self) -> None:
        from azure.cosmos import CosmosClient, PartitionKey
        s = get_settings()
        client = CosmosClient.from_connection_string(s.cosmos_connection_string)
        db = client.create_database_if_not_exists(id=s.cosmos_database)
        self._c = db.create_container_if_not_exists(
            id=s.cosmos_container, partition_key=PartitionKey(path="/sessionId")
        )
    def append(self, session_id: str, message: str) -> int:
        import time
        import uuid
        self._c.upsert_item({"id": uuid.uuid4().hex, "sessionId": session_id,
            "message": message, "ts": time.time_ns()})
        rows = list(self._c.query_items(
            query="SELECT VALUE COUNT(1) FROM c WHERE c.sessionId=@s",
            parameters=[{"name": "@s", "value": session_id}], partition_key=session_id))
        return rows[0] if rows else 0
```

```

def recent(self, session_id: str, n: int = 5) -> list[str]:
    items = list(self._c.query_items(
        query="SELECT * FROM c WHERE c.sessionId=@s ORDER BY c.ts DESC",
        parameters=[{"name": "@s", "value": session_id}], partition_key=session_id))
    return [i["message"] for i in reversed(items[:n])]
@lru_cache
def get_store():
    return CosmosStore() if get_settings().use_cosmos else InMemoryStore()
# the Semantic Kernel native plugin
class MemoryPlugin:
    @kernel_function(name="remember", description="Store a message; returns the session turn count.")
    )
    def remember(self, session_id: str, message: str) -> str:
        return str(get_store().append(session_id, message))
    @kernel_function(name="recall", description="Return recent messages for a session as JSON.")
    def recall(self, session_id: str) -> str:
        return json.dumps(get_store().recent(session_id))
@lru_cache
def build_kernel() -> Kernel:
    kernel = Kernel()
    kernel.add_plugin(MemoryPlugin(), plugin_name="memory")
    return kernel
async def chat(req: ChatRequest) -> ChatResponse:
    kernel = build_kernel()
    turns_result = await kernel.invoke(
        plugin_name="memory", function_name="remember",
        arguments=KernelArguments(session_id=req.session_id, message=req.message),
    )
    turns = int(str(turns_result))
    recent_result = await kernel.invoke(
        plugin_name="memory", function_name="recall",
        arguments=KernelArguments(session_id=req.session_id),
    )
    recent = json.loads(str(recent_result))
    return ChatResponse(
        reply=f"Got it. I now remember {turns} message(s) in session '{req.session_id}'.",
        session_id=req.session_id,
        memory_turns=turns,
        recent=recent,
        mode="cosmos" if get_settings().use_cosmos else "mock",
        invoked_via="semantic-kernel native plugin",
    )

```

» main.py

```

"""Semantic Kernel + Azure Cosmos DB - Durable Conversation Memory (FastAPI service).
Real SK kernel + native memory plugin. Works offline (in-memory store); set
COSMOS_CONNECTION_STRING for durable Cosmos DB memory. Run: uvicorn app.main:app --reload
"""
from fastapi import FastAPI
from app.kernel_app import ChatRequest, ChatResponse, chat, get_settings
settings = get_settings()
app = FastAPI(title="Semantic Kernel + Azure Cosmos DB - Conversation Memory", version="0.1.0")
@app.get("/health", tags=["health"])
def health() -> dict[str, str]:
    return {
        "status": "ok",
        "framework": "semantic-kernel",
        "memory": "cosmos" if settings.use_cosmos else "mock",
    }
@app.get("/", tags=["root"])
def root() -> dict[str, str]:
    return {
        "service": "agentic-ai-azure-sk-cosmosdb",
        "endpoint": "/api/v1/chat",
        "memory": "cosmos" if settings.use_cosmos else "mock",
        "docs": "/docs",
    }
@app.post("/api/v1/chat", response_model=ChatResponse, tags=["sk-cosmosdb"])
async def chat_endpoint(payload: ChatRequest) -> ChatResponse:
    return await chat(payload)

```